

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ КОЛЛЕКТИВНОГО ОБМЕНА
УЧЕБНЫМИ МАТЕРИАЛАМИ СТУДЕНТОВ**

по направлению 09.03.01 Информатика и вычислительная техника

Образовательная программа (профиль) «Веб-технологии»

Студент: _____ / Петров Александр Игоревич, 221–321 /
подпись *ФИО*

Руководитель ВКР: _____ / Клейменова Лариса Маясовна, к.п.н. /
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

по направлению 09.03.01 Информатика и вычислительная техника Образовательная
программа (профиль) «Веб-технологии»

Тема ВКР	Веб-приложение для коллективного обмена учебными материалами студентов.
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Веб-приложение предназначено для организации и совместного использования учебных материалов студентами в рамках образовательных программ. Система позволяет централизованно хранить, структурировать и находить материалы по различным дисциплинам и вузам. Пользователи могут добавлять собственные материалы, просматривать и сохранять материалы других пользователей, формируя персональную базу знаний. Приложение направлено на упрощение доступа к учебной информации, повышение эффективности самостоятельного обучения и взаимодействия между студентами.
Основные функции	1. Осуществление регистрации и авторизации пользователей. 2. Иерархический каталог: университеты → дисциплины → посты. 3. Публикация постов с прикреплением до пяти изображений; редактирование и удаление собственных публикаций. 4. Раздел «Знания» для публикаций, не привязанных к конкретной дисциплине. 5. Система избранного с боковой панелью сохранённых постов.
Используемые технологии и платформы	Next.js, TypeScript, Tailwind CSS, Node.js, NestJS, REST API, PostgreSQL, Prisma, TanStack Query, React Hook Form

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Провести анализ предметной области веб-приложений для обмена материалами. 2. Выполнить анализ существующих аналогичных решений и определить их преимущества и недостатки. 3. Определить целевую аудиторию и пользовательские роли. 4. Сформировать функциональные требования к системе. 5. Разработать пользовательские сценарии (публикация, просмотр, навигация по каталогу, сохранение в избранное). 6. Спроектировать структуру и навигацию пользовательского интерфейса (прототипы/макеты страниц). 7. Спроектировать архитектуру клиент-серверного взаимодействия и REST API. 8. Спроектировать схему базы данных (пользователи, вузы, дисциплины, посты, вложения, избранное). 9. Реализовать серверную часть веб-приложения на NestJS/Node.js. 10. Реализовать клиентскую часть веб-приложения на Next.js/TypeScript с Tailwind CSS и TanStack Query. 11. Реализовать загрузку и хранение изображений, отображение галереи вложений. 12. Реализовать механизм избранного и иерархическую навигацию по каталогу. 13. Провести тестирование веб-приложения и устранить выявленные ошибки.
Состав технической документации	1. Техническое задание. 2. Пояснительная записка.
Состав графической части	1. Презентация: 1 экз. 2. Диаграмма IDEF0 AS-IS (анализ существующего процесса): 1 экз. 3. Диаграмма IDEF0 TO-BE (проектируемая система): 1 экз. 4. ERD-диаграмма базы данных: 1 экз. 5. Схема архитектуры веб-приложения (frontend/backend): 1 экз. 6. Схема взаимодействия компонентов системы (API, клиент, БД): 1 экз. 7. Пользовательские сценарии (Use Case / User Flow): 1 экз. 8. Макеты интерфейса веб-приложения: 1 экз.

ПЛАН РАБОТЫ НАД ВКР

Этапы	Недели семестра																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Провести анализ предметной области.																		
Сравнить существующие аналогичные решения.																		
Провести анализ целевой аудитории веб-приложения.																		
Определить функциональные требования к веб-приложению.																		
Разработать пользовательские сценарии.																		
Спроектировать архитектуру веб-приложения.																		
Разработать дизайн-макеты страниц и компонентов веб-приложения.																		
Спроектировать схему базы данных.																		
Разработать серверную часть веб-приложения.																		
Разработать клиентскую часть веб-приложения.																		
Разработать руководство по использованию.																		
Провести различные виды тестирования веб-приложения.																		
Обеспечить информационную безопасность веб-приложения.																		

РУКОВОДИТЕЛЬ ОП:

« ____ » _____ 2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

« ____ » _____ 2026, _____ / Клейменова Лариса Маясовна, к.п.н. /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

« ____ » _____ 2026, _____ / Петров Александр Игоревич, 221–321 /
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: веб-приложение для коллективного обмена учебными материалами студентов.

Цель работы: разработать веб-приложение, предназначенное для централизованного хранения и структурирования учебных материалов по вузам и дисциплинам, обеспечения удобной иерархической навигации по каталогу, а также формирования персональной базы знаний пользователей с использованием механизма избранного.

Объект исследования: веб-приложение, обеспечивающее студентам инструменты для публикации и просмотра учебных материалов в виде постов с изображениями, их классификации по образовательным организациям и дисциплинам, навигации по иерархическому каталогу, а также сохранения материалов в избранное для быстрого доступа.

Предмет исследования: процесс организации коллективного обмена учебными материалами в студенческой среде и автоматизация управления знаниями в формате веб-платформы.

Работа состоит из Введения, трех глав, Заключения, Списка использованных источников и Приложений. Общий объем работы составляет X страниц, включая Y страниц Приложений. В работе содержится A рисунков, B таблиц и Z листингов кода. Библиография включает N источников.

Во Введении изложены цель, задачи, объект и предмет исследования, актуальность, новизна и практическая значимость работы. Первая глава посвящена анализу предметной области систем обмена учебными материалами, описанию процессов публикации постов и навигации по каталогу, обзору существующих аналогов и определению требований к системе. Вторая глава описывает проектирование и реализацию веб-приложения: архитектуру клиент-серверного взаимодействия на базе NestJS и Next.js, разработку REST API, проектирование базы данных и пользовательского интерфейса, а также реализацию ключевых модулей (регистрация и авторизация, CRUD публикаций с загрузкой изображений, иерархический каталог, раздел «Знания», механизм избранного). Третья глава посвящена тестированию системы, вопросам информационной безопасности и авторизации, оценке удобства использования, а также возможностям масштабирования платформы (поддержка

нескольких вузов, расширение дисциплин, добавление поиска и модерации). В Заключении представлены выводы по выполненной работе и перспективы дальнейшего развития системы.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	9
1 ПРЕДМЕТНАЯ ОБЛАСТЬ И АНАЛИЗ ТРЕБОВАНИЙ	11
1.1 Анализ предметной и проблемной области	11
1.2 Анализ процесса публикации поста	13
1.3 Анализ процесса навигации по каталогу и просмотра публикаций	17
1.4 Обзор аналогов и существующих решений	21
1.5 Требования к системе	32
2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СИСТЕМЫ	33
2.1 Выбор технологического стека	34
2.2 Проектирование модели данных	36
2.3 Проектирование макетов пользовательского интерфейса	38
2.4 Реализация серверной части	44
2.5 Реализация клиентской части	48
3 ТЕСТИРОВАНИЕ И ОЦЕНКА СИСТЕМЫ	52
3.1 Сценарий тестирования для роли «Гость» — поиск учебного материала	54
3.2 Сценарий тестирования для роли «Студент (USER)» — публикация материала и работа с избранным	56
3.3 Сценарий тестирования для роли «Администратор (ADMIN)» — управление каталогом и модерация	57
3.4 Информационная безопасность	58
3.5 Масштабируемость и перспективы развития	62
ЗАКЛЮЧЕНИЕ	65
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	68
СЦЕНАРИЙ ВИДЕОРОЛИКА ДЛЯ ЗАЩИТЫ ВКР	70
Кейс	70
Хронометраж	70

ВВЕДЕНИЕ

Современная образовательная среда активно переходит к цифровым форматам обучения и совместной работы. При этом ключевая проблема для студентов сохраняется: учебные материалы часто распределены по разным источникам (чаты, облачные диски, личные заметки, файлы преподавателей), не имеют единой структуры и быстро теряются. В результате возрастает время на поиск нужной информации, снижается регулярность самостоятельной подготовки и усложняется обмен учебными материалами внутри учебной группы.

Традиционно обмен учебными материалами происходит через неформальные каналы: мессенджеры, социальные сети, общие папки и форумы. Данный подход неудобен из-за отсутствия единого каталога и механизмов классификации, ограниченных возможностей поиска, а также недостаточной прозрачности: сложно понять актуальность файла, к какой дисциплине он относится и насколько он полезен. Отдельной проблемой является отсутствие персонализации — даже если материал найден, его невозможно быстро сохранить «в свою базу» и вернуться к нему позже в несколько кликов.

Актуальность данной работы обусловлена необходимостью разработки веб-приложения, обеспечивающего централизованное хранение, структурирование и быстрый доступ к учебным материалам. Создание системы коллективного обмена учебными материалами позволит упорядочить материалы по вузам и дисциплинам, повысить скорость поиска и повторного использования информации, а также снизить потери учебных ресурсов при смене учебных групп или формата обучения. Кроме того, наличие механизма избранного позволит пользователям формировать персональную базу знаний и поддерживать систематичность подготовки.

Целью данной работы является разработка веб-приложения для коллективного обмена учебными материалами студентов с возможностью структурирования, поиска и сохранения материалов в избранное. Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести обзор аналогов систем обмена учебными материалами и определить их преимущества и недостатки.
2. Выполнить анализ предметной области и выделить основные сценарии использования (поиск, публикация, просмотр, сохранение в избранное).
3. Определить целевую аудиторию веб-приложения и пользовательские роли.
4. Сформировать функциональные и нефункциональные требования к системе.

5. Спроектировать структуру и навигацию пользовательского интерфейса, подготовить макеты ключевых экранов.
6. Спроектировать архитектуру веб-приложения и клиент-серверное взаимодействие.
7. Разработать модель данных и спроектировать схему базы данных.
8. Реализовать серверную часть веб-приложения (REST API, авторизация, работа с знаниями и избранным).
9. Реализовать клиентскую часть веб-приложения (каталог материалов, поиск и фильтрация, карточки, просмотр, профиль, избранное).
10. Провести тестирование разработанной системы и устранить выявленные недостатки.
11. Описать возможности масштабирования и дальнейшего развития системы (расширение классификаторов, модерация, рекомендации и др.).

Объект исследования — веб-приложение, предназначенное для организации коллективного обмена учебными материалами.

Предмет исследования — процесс структурирования, хранения и поиска учебных материалов и поддержка совместного использования знаний студентами в цифровой среде.

Таким образом, разработка веб-приложения позволит централизовать учебные материалы, обеспечить удобную классификацию по вузам и дисциплинам, ускорить поиск и доступ к информации, а также повысить эффективность самостоятельной подготовки за счет формирования персональной базы знаний через механизм избранного. Система может быть востребована в вузах и учебных сообществах, где важно быстрое распространение актуальных материалов и поддержка регулярной учебной практики.

1 ПРЕДМЕТНАЯ ОБЛАСТЬ И АНАЛИЗ ТРЕБОВАНИЙ

1.1 Анализ предметной и проблемной области

Предметная область данного исследования охватывает разработку веб-приложений в сфере цифрового образования, ориентированных на организацию, хранение и распространение учебных материалов между студентами. Данное направление относится к образовательным технологиям (EdTech) и связано с цифровизацией учебного процесса, развитием онлайн-форматов и ростом потребности в удобных инструментах доступа к знаниям.

В реальных условиях учебные материалы часто распределены по множеству источников: мессенджеры, социальные сети, облачные хранилища, личные файлы студентов и преподавателей. Такая фрагментация приводит к потере материалов, отсутствию единой структуры и сложностям в поиске. Особенно остро проблема проявляется при подготовке к экзаменам и зачётам, когда требуется быстро восстановить доступ к конспектам, методическим указаниям, лабораторным работам и типовым заданиям по конкретной дисциплине.

Сложившаяся практика обмена учебными материалами через неформальные каналы имеет ряд ограничений. Во-первых, отсутствует удобная классификация: файлы редко систематизируются по вузам и дисциплинам, а названия и содержимое не стандартизированы. Во-вторых, возможности поиска ограничены: даже при наличии общего облачного диска пользователь вынужден просматривать множество папок и файлов. В-третьих, отсутствует персонализация: студент не может быстро сформировать собственный набор полезных материалов и возвращаться к нему без повторного поиска.

Основной проблемой предметной области является отсутствие единой платформы, обеспечивающей структурированное хранение учебных материалов, быстрый поиск по атрибутам (вуз, дисциплина, тип материала) и удобные механизмы повторного доступа к найденным материалам. При ручной организации материалов (через папки и чаты) неизбежно возникает дублирование, теряется актуальность, снижается качество навигации и возрастает время на подготовку. Это негативно влияет на эффективность самостоятельного обучения и обмена учебными материалами внутри учебного сообщества.

Современные веб-технологии позволяют решать перечисленные задачи за счёт создания централизованных систем управления контентом. Использование клиент-серверной архитектуры, механизмов авторизации, базы данных и инструментов поиска обеспечивает удобный доступ к материалам, поддержку добавления и редактирования контента, а также формирование персональных подборок через механизм избранного. В результате

снижается “стоимость поиска” информации, а доступ к знаниям становится быстрее и более предсказуемым.

Перспективы развития подобных платформ связаны с масштабированием классификаторов (поддержка нескольких вузов, расширение дисциплин и типов материалов), внедрением модерации контента, системой тегов и рекомендаций, а также анализом популярности материалов. Такие улучшения повышают качество базы знаний и делают обмен учебными материалами более безопасным и удобным для пользователей.

Таким образом, разработка веб-приложения как системы коллективного обмена учебными материалами направлена на решение актуальной задачи: обеспечение структурированного хранения знаний, удобного поиска и быстрого повторного доступа к материалам. Это способствует повышению эффективности самостоятельной подготовки, улучшению взаимодействия студентов и упорядочиванию учебной информации в цифровой среде.

1.2 Анализ процесса публикации поста

Процесс публикации поста в системе представляет собой последовательность действий авторизованного пользователя, направленных на добавление новой текстовой публикации с изображениями в общую базу знаний в рамках конкретного университета и дисциплины. В отличие от неформального обмена материалами через мессенджеры и облачные хранилища, публикация в системе структурирует контент по иерархии «университет → дисциплина → пост», обеспечивает постоянный доступ к материалам для всех пользователей и поддерживает прикрепление изображений непосредственно к публикации.

В системе поддерживается два типа публикаций. Первый тип — тематические посты, привязанные к конкретной дисциплине университета и образующие структурированный каталог. Второй тип — посты раздела «Знания», не привязанные к конкретному университету или дисциплине; они образуют общую базу статей и заметок, доступных всем пользователям.

1.2.1 Участники процесса и роли

В процессе публикации поста участвуют следующие роли:

1. Авторизованный пользователь (студент) — создаёт публикацию: заполняет заголовок, текст и при необходимости прикрепляет до пяти изображений; может редактировать и удалять собственные посты.
2. Система (веб-приложение) — обеспечивает форму ввода, валидацию данных, загрузку изображений на сервер, создание записей в базе данных и отображение публикаций в каталоге.
3. Администратор/модератор — может удалять любые публикации для поддержания качества контента.

1.2.2 Входные данные и выходные результаты процесса

Входными данными процесса публикации являются:

1. Заголовок публикации — краткое название, отражающее содержание поста.
2. Основной текст — развёрнутое содержание публикации (конспект, описание, методика и т. п.).
3. Изображения — до пяти файлов формата image (фотографии, схемы, скриншоты, отсканированные материалы).
4. Навигационный контекст — идентификаторы университета и дисциплины (для тематических постов) либо раздел «Знания».

Выходными результатами процесса являются:

1. Новая запись публикации в базе данных с привязкой к дисциплине (или разделу «Знания»), автору и дате создания.
2. Записи вложений для каждого загруженного изображения с URL-путём к файлу на сервере.
3. Доступность публикации для просмотра всеми пользователями в соответствующем разделе каталога.

1.2.3 Этап 1. Проверка авторизации и инициация публикации

Процесс начинается с того, что пользователь переходит к нужной дисциплине (или разделу «Знания») и нажимает кнопку «Добавить пост». Данная кнопка отображается только авторизованным пользователям: сервер проверяет наличие `access_token` в `cookie`; если токен отсутствует, пользователь перенаправляется на страницу входа. После успешной авторизации пользователь возвращается к форме добавления публикации.

1.2.4 Этап 2. Заполнение формы публикации

Ключевым отличием системы от обмена файлами через чат является обязательное структурирование данных. Пользователь заполняет:

1. Заголовок. Название должно кратко и информативно отражать содержание поста (например: «Лекция 3: алгоритмы сортировки», «Решения типовых задач по теореме Ху»).
2. Текст публикации. Развёрнутое содержание: конспект, методические указания, разбор задач и т. п.
3. Изображения (необязательно). Пользователь может прикрепить до пяти изображений — схемы, чертежи, отсканированные лекции, скриншоты с задачами. Система принимает только файлы с MIME-типом `image/`

Форма выполняет клиентскую валидацию: проверяет заполненность обязательных полей. При ошибках подсвечиваются проблемные поля с соответствующими сообщениями.

1.2.5 Этап 3. Отправка данных и сохранение на сервере

После заполнения формы пользователь нажимает кнопку «Добавить». Данные отправляются в формате `multipart/form-data` через Next.js API-маршрут, который проксирует запрос на бэкенд NestJS (эндпоинт `POST /subjects/:subjectId/posts` или аналогичный для раздела «Знания»). Маршрут передаёт тело запроса как `Buffer` с исходным заголовком `Content-Type`, сохраняя корректную `multipart`-границу.

На сервере выполняется следующая последовательность действий:

1. `AuthGuard` проверяет JWT-токен из заголовка `Authorization` и извлекает данные пользователя.

2. FilesInterceptor (multer) обрабатывает прикрепленные изображения: проверяет MIME-тип (принимаются только image/), сохраняет файлы в папку uploads/ с именем вида {timestamp}-{random}.{ext}.
3. PostsService создаёт запись Post в базе данных: заголовок, текст, идентификаторы дисциплины и пользователя, дата создания.
4. Для каждого загруженного изображения создаётся запись Attachment с URL вида /uploads/имя-файла.расширение, связанная с созданным постом.
5. Сервер возвращает созданный объект публикации.

1.2.6 Этап 4. Отображение результата и навигация

После успешной публикации пользователь перенаправляется обратно на страницу дисциплины (или раздела «Знания»), где новый пост отображается в виде карточки: заголовок, предпросмотр текста и первое изображение (если есть). Для просмотра полного содержания и всех изображений пользователь переходит на страницу поста, где доступна галерея изображений.

1.2.7 Этап 5. Редактирование и удаление публикации

Авторизованный владелец поста может редактировать или удалить публикацию непосредственно со страницы просмотра. Редактирование позволяет изменить заголовок и текст, удаление — полностью убрать пост и все связанные вложения. Данные действия также доступны модераторам и администраторам для контроля качества контента.

1.2.8 Этап 6. Обработка ошибок

В процессе публикации возможны следующие ошибки:

1. Отказ в доступе (пользователь не авторизован) — перенаправление на страницу входа.
2. Ошибки валидации (пустые обязательные поля) — подсветка полей и вывод сообщений.
3. Недопустимый тип файла (не изображение) — сервер отклоняет файл и возвращает ошибку.
4. Сбой загрузки (ошибка сети или сервера) — вывод уведомления; введенные данные сохраняются в форме.

1.2.9 Итоговое описание процесса

Таким образом, процесс публикации поста включает: проверку авторизации, заполнение формы (заголовок, текст, изображения), отправку данных через Next.js API-маршрут на бэкенд NestJS, сохранение поста и вложений в базе данных и файловой системе, а также отображение публикации в каталоге. Иерархическая привязка «университет → дисциплина → пост»

обеспечивает структурированный доступ к материалам, а поддержка изображений позволяет публиковать наглядные материалы — конспекты, схемы и решения задач.

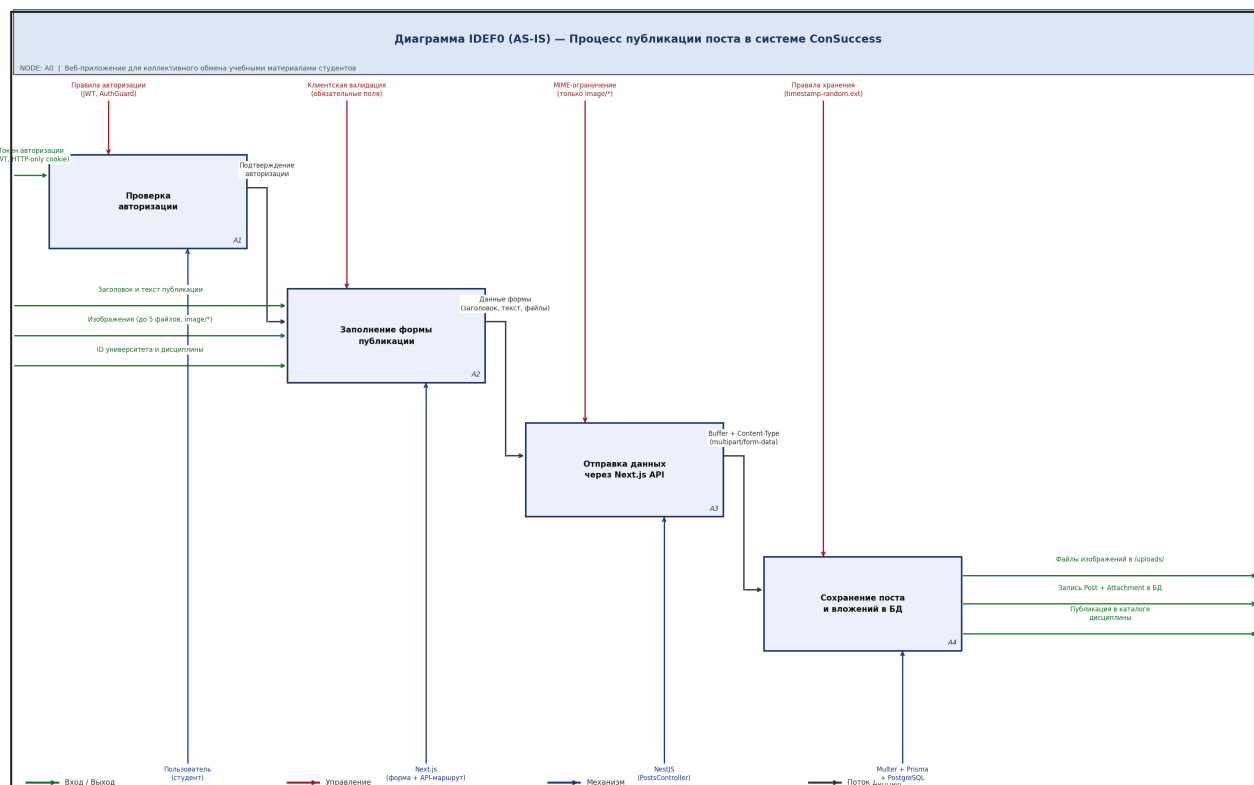


Рисунок 1: Диаграмма процесса публикации поста IDEF0 (as is)

1.3 Анализ процесса навигации по каталогу и просмотра публикаций

Процесс навигации по каталогу и просмотра публикаций в системе направлен на предоставление пользователям удобного и предсказуемого доступа к учебным материалам через иерархическую структуру. Система использует подход структурной навигации: каталог организован по принципу «университет → дисциплина → посты», что позволяет пользователям последовательно сужать область просмотра до релевантного учебного контекста без необходимости вводить поисковые запросы.

Помимо основной иерархии, в системе предусмотрен отдельный раздел «Знания», агрегирующий публикации общего характера, не привязанные к конкретному университету или дисциплине. Дополнительным механизмом доступа к материалам является система избранного, позволяющая пользователям сохранять нужные посты и возвращаться к ним через боковую панель без повторной навигации по каталогу.

1.3.1 Участники процесса и роли

В процессе навигации и просмотра участвуют следующие роли:

1. Пользователь (авторизованный или гость) — просматривает каталог, переходит по уровням иерархии, открывает страницы постов, добавляет посты в избранное.
2. Система (веб-приложение) — обрабатывает запросы к серверным компонентам, получает данные через API, формирует страницы с карточками и деталями публикаций.
3. Администратор/модератор — может удалять нежелательный контент, что влияет на состав каталога.

1.3.2 Структура каталога и уровни навигации

Основная навигация в системе строится по трём уровням иерархии:

1. Уровень 1 — Список университетов (/universities). Отображает карточки всех доступных университетов (логотип, название, город). Пользователь выбирает нужный университет для перехода к его дисциплинам.
2. Уровень 2 — Дисциплины университета (/universities/[id]). Отображает карточки дисциплин, прикрепленных к выбранному университету. Пользователь выбирает дисциплину для просмотра публикаций.
3. Уровень 3 — Посты дисциплины (/universities/[id]/subjects/[subjectId]). Отображает список постов в виде карточек: заголовок, краткий предпросмотр текста, первое изображение (если есть). Пользователь открывает конкретный пост для просмотра полного содержания.

Дополнительно доступен раздел «Знания» — список общих публикаций, не привязанных к конкретной дисциплине. Главная страница системы отображает недавние посты из всех разделов, обеспечивая быстрый доступ к свежим публикациям.

1.3.3 Входные данные и выходные результаты процесса

Входными данными процесса навигации являются:

1. Действия пользователя — переходы по ссылкам, нажатие кнопок карточек, клики по кнопке избранного.
2. Параметры маршрута — идентификаторы университета, дисциплины, поста в URL-адресе страницы.
3. Токен авторизации — определяет доступность кнопки добавления в избранное и отображение действий с постом (редактирование, удаление).

Выходными результатами процесса являются:

1. Список карточек (университеты, дисциплины или посты) на соответствующем уровне каталога.
2. Полная страница поста с текстом, галереей изображений и данными об авторе.
3. Актуальный список избранного в боковой панели.

1.3.4 Этап 1. Вход в каталог и выбор университета

Пользователь переходит на страницу списка университетов. Серверный компонент запрашивает данные через API и отображает карточки университетов в сетке. Пользователь выбирает нужный университет и переходит к списку его дисциплин. Кнопка «← Назад» обеспечивает возврат к предыдущему уровню.

1.3.5 Этап 2. Выбор дисциплины

На странице университета отображаются карточки всех дисциплин данного учебного заведения. Пользователь выбирает нужную дисциплину и переходит к списку публикаций по ней.

1.3.6 Этап 3. Просмотр списка публикаций

На странице дисциплины отображаются карточки постов. Каждая карточка содержит:

1. Заголовок публикации.
2. Краткий предпросмотр текста (первые несколько строк).
3. Первое прикрепленное изображение (если есть), с режимом object-contain для отображения без обрезки.
4. Ссылку «Открыть →» для перехода к полному тексту.

1.3.7 Этап 4. Просмотр публикации

Страница публикации содержит полный текст поста, заголовок, данные об авторе, дату создания и галерею изображений. Галерея реализована как клиентский компонент с возможностью открытия изображения в полноэкранном режиме (lightbox). Авторизованному владельцу поста доступны кнопки редактирования и удаления. Кнопка «← Назад» возвращает пользователя к списку постов дисциплины.

1.3.8 Этап 5. Использование механизма избранного

На карточке и странице каждого поста отображается кнопка-звёздочка. Авторизованный пользователь может добавить пост в избранное или убрать его оттуда. Все сохранённые посты отображаются в боковой панели («Избранное»), доступной на всех страницах авторизованной зоны, в виде списка ссылок. Это позволяет пользователям формировать персональную подборку нужных материалов и возвращаться к ним без повторной навигации по каталогу.

1.3.9 Этап 6. Обработка ошибок и исключительных ситуаций

При навигации возможны следующие ошибки:

1. Несуществующий идентификатор (университет, дисциплина или пост не найден) — Next.js возвращает страницу 404.
2. Ошибка сети или недоступности API — отображается соответствующее сообщение об ошибке.
3. Попытка добавить в избранное без авторизации — перенаправление на страницу входа.

1.3.10 Итоговое описание процесса

Таким образом, навигация по каталогу строится по принципу иерархического сужения: пользователь последовательно переходит от списка университетов к дисциплинам и затем к публикациям по конкретному предмету. Раздел «Знания» обеспечивает доступ к материалам общего характера, а механизм избранного позволяет формировать персональную подборку для быстрого повторного доступа. Данный подход обеспечивает предсказуемую навигацию и соответствует привычной модели обучения, организованной по учебным заведениям и дисциплинам.

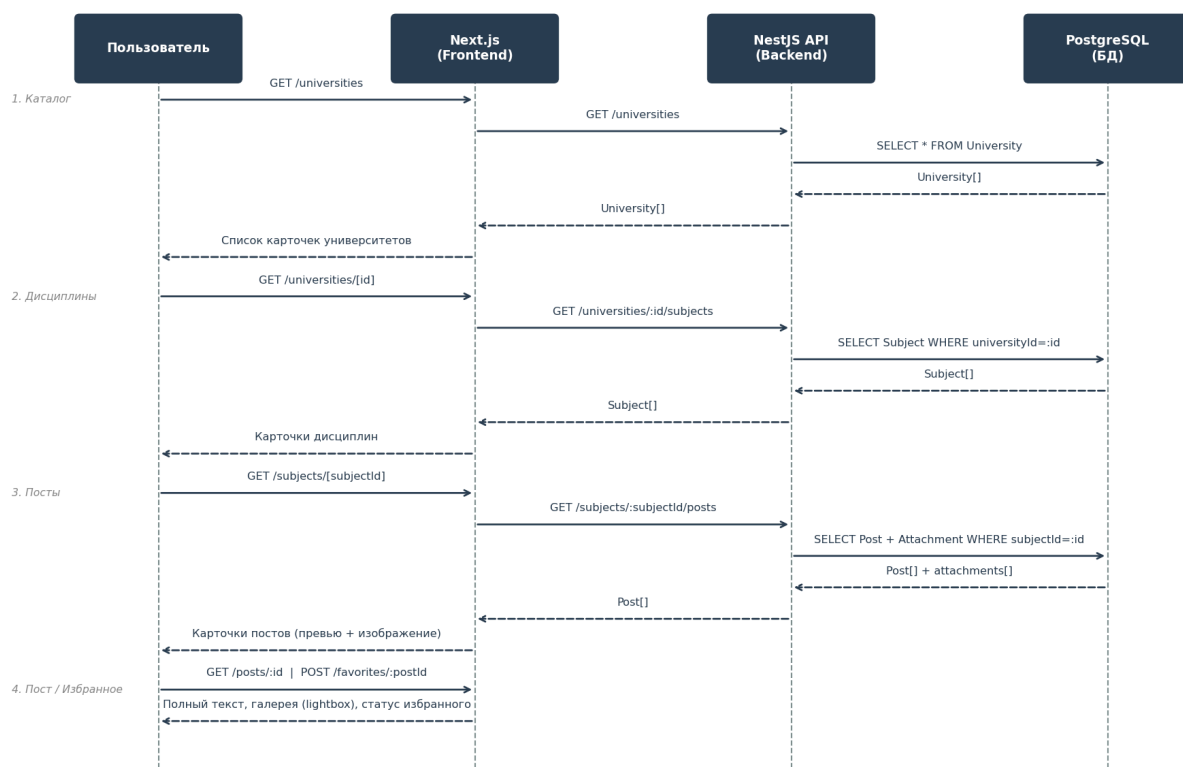


Рисунок 2: UML(sequence)-диаграмма процесса навигации по каталогу и просмотра публикаций

1.4 Обзор аналогов и существующих решений

Современный рынок образовательных технологий активно развивается, расширяя спектр цифровых решений для обучения и самообразования. При этом всё большее значение приобретают сервисы, ориентированные не только на проведение занятий, но и на удобную организацию учебных материалов: их структурирование, быстрый поиск и повторный доступ. Для студентов особенно актуальны инструменты, которые помогают снизить время на “поиск по чатам и папкам” и превращают разрозненные файлы в упорядоченную базу знаний.

Для определения оптимального набора функций и ключевых аспектов, требующих особого внимания при разработке и проектировании веб-приложения, необходимо проанализировать существующие решения, представленные на рынке. В рамках исследования были отобраны наиболее распространённые платформы, используемые студентами для обмена материалами, а также системы, применяемые образовательными организациями для размещения учебного контента. Рассматриваемые решения различаются по назначению и модели использования: от публичных библиотек конспектов и файловых каталогов до корпоративных систем, где материалы размещаются внутри конкретных курсов и доступны ограниченному кругу пользователей.

Анализ аналогов позволяет выявить наиболее удачные практики: каталогизацию материалов по дисциплинам и учебным организациям, развитые механизмы поиска и фильтрации, наличие персональных подборок (избранного), а также возможности управления контентом и контроля качества. Одновременно становятся заметны типовые ограничения: фрагментарность источников, отсутствие единого стандарта описания материалов, сложности с оценкой актуальности и полезности, а также недостаточная персонализация доступа. Таким образом, обзор существующих решений является необходимым этапом для формирования требований к разрабатываемой системе и обоснования проектных решений.

- 1) Notion — универсальная платформа для ведения заметок и построения баз знаний в формате “рабочего пространства”. По сути это конструктор страниц, где можно хранить текст, таблицы, файлы и собирать всё в базы данных. Студенты обычно делают в Notion “рабочую тетрадь” на семестр: страницы по дисциплинам, конспекты лекций, чек-листы, ссылки на материалы, иногда — базы “Дисциплины/Темы/Материалы” с фильтрами.

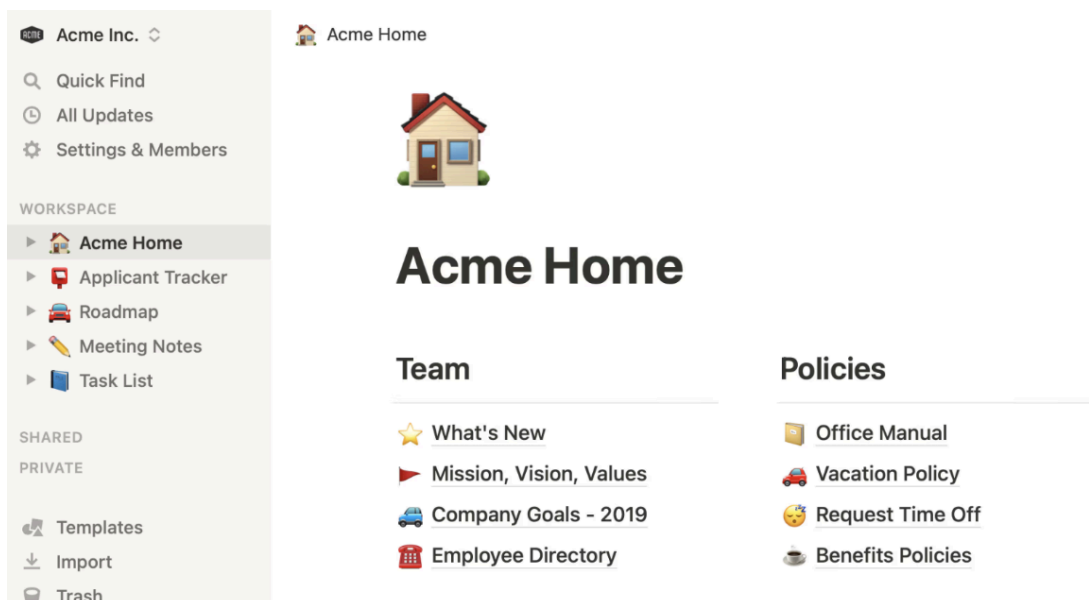


Рисунок 3: Интерфейс приложения Notion

Релевантные функции для предметной области

1. **Структурирование:** древовидные страницы и базы данных (таблица/доска/календарь) с настраиваемыми полями (теги, курс, преподаватель и т.п.).
2. **Поиск:** глобальный поиск по workspace для быстрого нахождения материалов по ключевым словам.
3. **Фильтры и представления:** возможность создавать разные представления (например, “Экзамен”, “Лабы”, “По курсу”) и комбинировать условия фильтрации.
4. **Совместная работа:** общий доступ по ссылке/приглашению, комментарии, упоминания пользователей.
5. **Хранение контента:** вложения, ссылки и встраивания (при этом сервис не является “медиахранилищем” в классическом смысле).
6. **Шаблоны:** стандартизация карточки материала (например: “Название → дисциплина → тип → ссылка → описание”).

Сильные стороны

1. **Гибкая структура и кастомизация:** позволяет быстро собрать “базу материалов” под свои потребности без разработки.
2. **Удобный UX базы знаний:** страницы, теги, фильтры, шаблоны и быстрый поиск упрощают навигацию и поддержку порядка.
3. **Поддержка коллективной работы:** комментарии и механизмы шаринга приближают Notion к сценарию “коллективного обмена”.

Ограничения как платформы материалов для студентов

1. **Нет доменной модели “вуз → дисциплина → материал” из коробки:** структуру приходится вручную строить через поля и базы.
2. **Не публичный каталог по умолчанию:** Notion ориентирован на workspace, а не на витрину материалов для широкой аудитории студентов.
3. **Нет контроля качества и актуальности:** отсутствуют встроенные механизмы модерации, стандарты публикации и единая “карточка материала”, как у специализированных сервисов.
4. **Слабее сценарии массового поиска:** сервис эффективен внутри одного пространства, но плохо подходит для каталога по множеству вузов/дисциплин.
5. **Избранное ограничено уровнем страниц/закладок:** это не всегда эквивалентно “избранному материалов” как отдельной функции каталога.

Выводы для требований к системе

1. **Стоит перенять:** карточную модель материала, теги, фильтры, шаблонность, быстрый поиск и аккуратный UX базы знаний.
 2. **Важно реализовать отличия:** строгую классификацию (вуз/дисциплина/тип), публичный или полупубличный каталог, “избранное” как отдельный механизм, а при необходимости — модерацию и правила публикации.
- 2) YoNote — онлайн-платформа для ведения учебных конспектов и организации материалов в цифровом виде. Сервис ориентирован на создание заметок, их структурирование по темам и быстрый доступ к информации в процессе обучения. В типичном сценарии студент использует YoNote как “электронную тетрадь”: создаёт конспекты по дисциплинам, закрепляет важные фрагменты, прикрепляет ссылки/файлы, ведёт списки задач и подготовку к зачётам/экзаменам. Материалы обычно группируются по предметам и темам, что упрощает повторение и поиск.

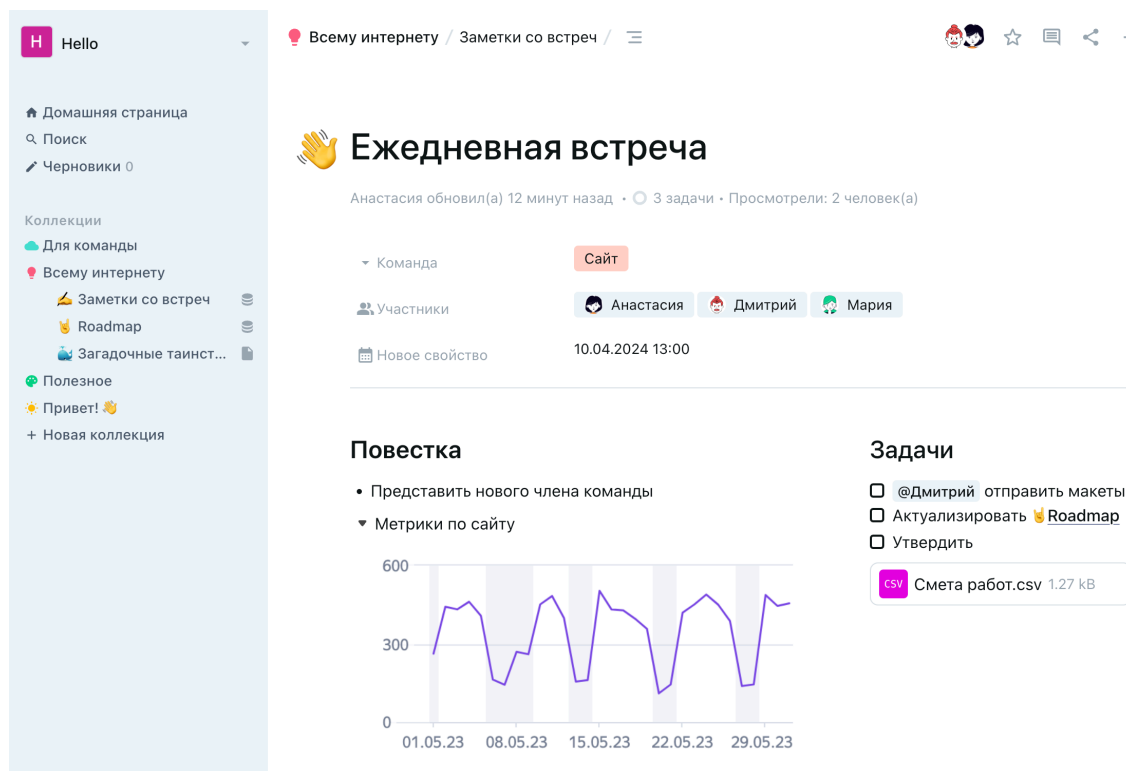


Рисунок 4: Интерфейс платформы YoNote

Релевантные функции для предметной области:

1. **Создание и ведение заметок:** поддержка форматирования, структурирование конспектов и фиксация ключевых сведений по дисциплинам.
2. **Организация материалов:** группировка по предметам/темам, использование разделов/категорий для навигации по учебному контенту.
3. **Поиск:** поиск по заметкам и содержимому для быстрого восстановления нужных фрагментов и тем.
4. **Работа с вложениями и ссылками:** добавление файлов и внешних ресурсов как дополнения к конспектам.
5. **Доступ с разных устройств:** использование веб-доступа/синхронизации для работы с материалами в любом месте.
6. **Обмен материалами:** возможность делиться заметками (по ссылке или через доступ) — как базовый механизм обмена.

Сильные стороны:

1. **Фокус на учебных сценариях:** сервис подходит для регулярного ведения конспектов и подготовки к занятиям, помогает поддерживать порядок в материалах.
2. **Быстрый доступ к информации:** структурирование и поиск позволяют оперативно находить нужные темы и фрагменты.

3. **Удобство накопления знаний:** заметки, вложения и ссылки формируют единый контекст по дисциплине и облегчают повторение.

Ограничения как платформы материалов для студентов:

1. **Ориентация на личные заметки, а не на каталог:** даже при наличии обмена сервис чаще выступает как персональная “тетрадь”, а не как публичная витрина материалов.
2. **Нет строгой доменной модели каталога:** обычно отсутствует предопределённая структура вида “вуз → дисциплина → материал” с едиными правилами классификации.
3. **Ограниченные сценарии массового поиска:** поиск эффективен внутри личного пространства/группы, но сложнее масштабируется на большую аудиторию и множество дисциплин.
4. **Не всегда есть стандартизация карточки материала:** публикация материалов может быть разноформатной, что усложняет сравнение, оценку качества и повторное использование.
5. **Механизмы избранного/коллекций могут быть ограничены:** если есть закладки или списки, они не всегда реализованы как отдельная функция “избранного материалов” в каталоге.

Выводы для требований к системе:

1. **Стоит перенять:** удобное ведение заметок, структурирование по темам, быстрый поиск, работу с вложениями/ссылками и доступ с разных устройств.
 2. **Важно реализовать отличия:** каталогизацию по вузам/дисциплинам и типам материалов, “карточку материала” с метаданными, полноценное “избранное” и масштабируемый поиск по большому массиву публикаций, при необходимости — модерацию/правила публикации.
- 3) StuDocu — онлайн-платформа для обмена учебными материалами, где студенты загружают и находят конспекты, краткие конспекты, шпаргалки и материалы для подготовки к экзаменам. Доступ к материалам обычно организован через выбор учебного заведения и курса/дисциплины, чтобы находить документы, релевантные конкретной программе обучения. Типичный сценарий использования: студент выбирает свой вуз и курс, ищет нужную дисциплину, просматривает доступные документы и при необходимости загружает собственные материалы. На платформе также используется модель “вознаграждения” за публикацию (дни Premium-доступа) и есть

инструменты подготовки на базе загруженных материалов (например, краткие выжимки и тренировочные вопросы).

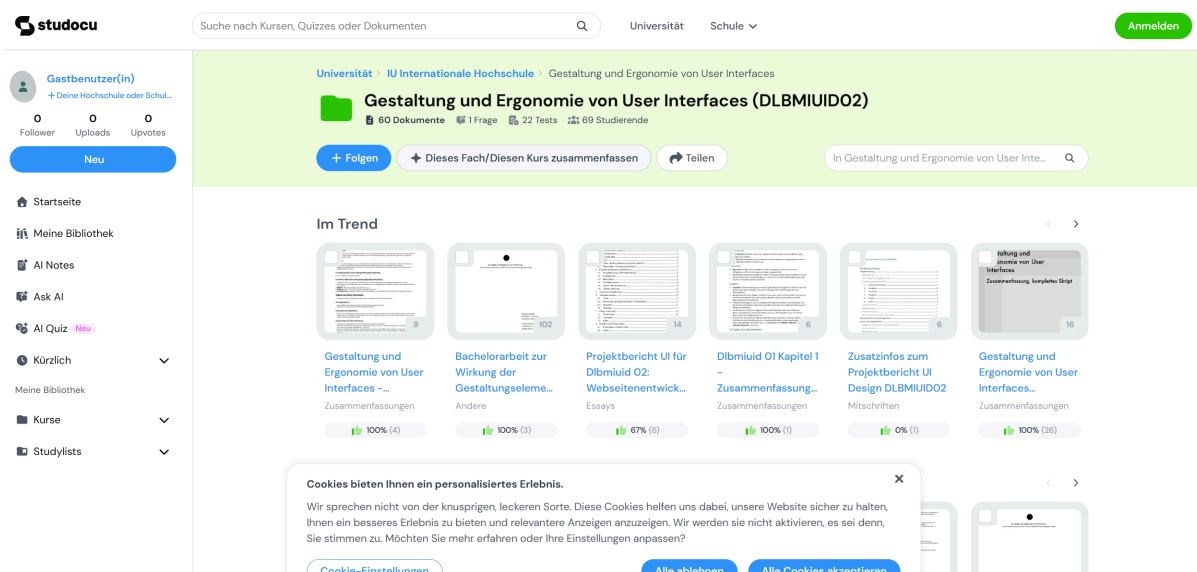


Рисунок 5: Интерфейс платформы StuDocu

Релевантные функции для предметной области:

1. **Каталог и поиск по учебному контексту:** навигация и поиск материалов через учебное заведение и курс/дисциплину.
2. **Публикация материалов:** загрузка учебных документов в поддерживаемых форматах и добавление базовых сведений о документе.
3. **Просмотр и скачивание:** предпросмотр материалов и скачивание (часть функций/контента может быть ограничена).
4. **Механизм мотивации публикации:** получение Premium-доступа за загрузку и принятие документов.
5. **Инструменты подготовки:** преобразование загруженных материалов в учебные форматы (например, краткие конспекты, тестовые вопросы).

Сильные стороны:

1. **Каталог “от студентов для студентов”:** большой объём пользовательских материалов по конкретным дисциплинам и курсам.
2. **Ориентация на учебный поиск:** удобно искать материалы под конкретный курс/программу, а не “абстрактно по теме”.
3. **Низкий порог участия:** пользователь может быстро пополнять библиотеку и получать выгоды за публикацию материалов.

4. **Усиление подготовки к экзаменам:** наличие инструментов, которые помогают “переварить” загруженные документы в более удобный формат.

Ограничения как платформы материалов для студентов:

1. **Неоднородное качество контента:** материалы пользовательские, поэтому полнота и корректность зависят от автора и источника.
2. **Риски авторских прав:** возможны загрузки материалов, на которые у пользователей нет прав; требуется модерация и процедура удаления по жалобам.
3. **Ограничения доступа:** часть популярного контента и/или расширенные функции могут быть доступны только через Premium.
4. **Слабая стандартизация “карточки материала”:** метаданные часто минимальны, из-за чего сложнее сравнивать материалы и контролировать качество.

Выводы для требований к системе:

1. **Стоит перенять:** идею каталога по учебному контексту (вуз/дисциплина), сильный поиск и фильтрацию, удобный сценарий публикации и мотивацию участников.
 2. **Важно реализовать отличия:** строгую доменную модель “вуз → дисциплина → материал” с обязательными метаданными, прозрачные правила публикации и лицензирования, модерацию/жалобы, а также “избранное” как отдельный механизм внутри каталога.
- 4) StudFiles — файловый архив учебных материалов, в котором контент публикуется пользователями и группируется по учебному контексту: вузам и дисциплинам. Платформа ориентирована на хранение и распространение документов (конспекты, методички, лабораторные, курсовые и т.п.) с возможностью быстрого просмотра и скачивания. Типичный сценарий использования: студент выбирает свой вуз и предмет, просматривает список доступных файлов, ищет нужный документ по названию и при необходимости загружает собственные материалы в соответствующий раздел. Для ускорения публикации предусмотрена пакетная загрузка (например, архивом с последующей распаковкой).

Фильтр

В каталоге: 1316 вузов, 5364 предметов, 8 064 702 файлов на 6937 55 Гб

Gold Coast, Australia	Костанай	Петрозаводск
Bond Uni Bond University 1 136	КТГУ Костанайский Государственный университет им. Ахмета Байтурсынова 444	КГПА Карельская государственная педагогическая академия 568
Nitra	Кострома	ПКГ Петрозаводская государственные консерватория им. А.К. Глазунова 90
UKF Univerzita Konstantina Filozofa 111	КТГУ Костромской государственный университет 4 354	ПетрГУ Петрозаводский Государственный Университет 5 110
Riga	Краматорск	Петропавловск
NMC Novikontas Maritime College 4 283	ДГМА Донбасская государственная машиностроительная академия 2 604	СКГУ Северо-Казахстанский государственный университет им. М. Козыбаева 936
Termiz	ДонНАСА Донбасская национальная академия строительства и архитектуры 2 428	Петропавловск: Камчатский
TISU Termiz Iqtisodiyot va Servis universiteti 13	Красноармейск	КамчатГТУ Камчатский государственный технический университет 124
Абакан	КИИ ДонНТУ Красноармейский индустриальный институт ДонНТУ 207	Печора
ХГУ Хакасский государственный университет им. Н.Ф.Катанова 2 175	Краснодар	ПГЭТ Печорский промышленно-экономический техникум 194
ХТИ Хакасский технический институт (филиал СФУ) 324	КТУКИ Краснодарский государственный университет культуры и искусств 669	Пинск
Актау	ККБМК Краснодарский Краевой Базовый Медицинский Колледж 3	ПГПТМ Пинский государственный профессионально-технический колледж машиностроения 98
КТУТИ Каспийский государственный университет технологий и инженерии им. Есенова 882	КубГУА Кубанский государственный аграрный университет 2 299	Полтава
Актобе	КубГМУ Кубанский государственный медицинский университет 4 197	Полтава
АРГУ Актюбинский региональный государственный университет им. К. Жубанова 898	КубГТУ Кубанский государственный технологический университет 10 789	ПГАА Полтавская государственная аграрная академия 1 232
ЗКГМУ Западно-Казахстанский государственный медицинский университет им. М.Оспанова 631	КубГУ Кубанский Государственный Университет 10 355	ПНПУ Полтавский национальный педагогический университет им. В.Г. Корженева 2 577
Алматы	КТУФХСТ Кубанский государственный университет физической культуры, спорта и туризма 429	ПолнНТУ Полтавский национальный технический университет им. Ю. Кондратюка 3 269
ALMU MAB Almaty Management University 912	КСЭИ Кубанский социально-экономический институт 85	ПУЭТ Полтавский университет экономики и торговли 1 550
АГКУЭЭТ Алматинский государственный колледж энергетики и электронных технологий 3 953	СГА Современная Гуманитарная Академия 2 622	УМСА Украинская медицинская стоматологическая академия 1 510

Рисунок 6: Интерфейс платформы StudFiles

Релевантные функции для предметной области:

1. **Каталог по вузам и дисциплинам:** навигация по структуре «вуз → предмет → файлы», что упрощает поиск материалов для конкретной учебной программы.
2. **Публикация материалов:** добавление файлов в выбранный предмет, включая пакетную загрузку (нескольких файлов за один раз).
3. **Поиск и фильтрация:** поиск по названию файла и базовые фильтры в каталоге для ускорения подбора материалов.
4. **Просмотр и скачивание:** просмотр страницы материала и скачивание файла для дальнейшего использования.

Сильные стороны:

1. **Ориентация на учебный контекст:** структура каталога через вузы и предметы близка к сценариям студентов и их учебным планам.
2. **Большой объём пользовательских материалов:** наличие множества файлов по разным дисциплинам повышает шанс быстро найти нужный документ.
3. **Простой вход в публикацию:** загрузка материалов не требует сложного оформления и позволяет быстро пополнять архив.

Ограничения как платформы материалов для студентов:

1. **Неоднородное качество и актуальность:** контент пользовательский, поэтому возможны ошибки, дубли и устаревшие версии.

2. **Слабая стандартизация описания материалов:** метаданные часто ограничены, из-за чего сложнее оценивать релевантность до скачивания и сравнивать материалы между собой.
3. **Полнотекстовый поиск может быть ограничен:** быстрый поиск часто сводится к названию файла, а поиск по содержимому зависит от индексации.
4. **Нет явной доменной модели “карточки материала”:** отсутствует единый формат публикации (тип, авторство, проверка, источники), что усложняет контроль качества.

Выводы для требований к системе:

1. **Стоит перенять:** каталогизацию по учебному контексту (вуз/дисциплина), простой сценарий публикации, предпросмотр и доступность материалов.
2. **Важно реализовать отличия:** “карточку материала” с обязательными метаданными (вуз/дисциплина/тип/автор/год/описание), встроенный поиск и фильтры по метаданным, “избранное” как отдельный механизм, а также правила публикации и модерацию/жалобы при необходимости.

Таблица 1 – Сравнение аналогов

Критерий	Notion	Yonote	StuDocu	StudFiles
Тип решения	Конструктор workspace: страницы + базы данных (БЗ/документация).	Командная база знаний: документы + базы данных/представления.	Публичный каталог учебных материалов (UGC) + инструменты подготовки.	Файловый архив студентов (UGC) с каталогом по вузам/предметам.
Каталогизация (вуз/дисциплина)	Нет «вуз—дисциплина» из коробки: задаётся вручную (страницы/поля/БД).	Нет «вуз—дисциплина» из коробки: задаётся вручную (документы/БД).	Каталог/поиск по университету и курсу/дисциплине.	Каталог «вузы → предметы → файлы».
Публикация материалов	Создание страниц/БД внутри workspace; публикация вовне — через доступ/ссылку.	Создание документов/БД внутри пространства; обмен — через доступ/ссылку.	Загрузка документов пользователями в каталог; доступ может быть частично ограничен.	Загрузка файлов в выбранный предмет; возможна пакетная загрузка (архивом).
Поиск и фильтры	Глобальный поиск по workspace + фильтры/представления в БД.	Поиск по базе знаний + представления БД (таблица/борд/календарь).	Поиск по каталогу и дисциплинам; ориентация на нахождение готовых документов.	Поиск/навигация в каталоге; фильтрация и выбор по вузу/предмету.
Публичность	По умолчанию закрыто: доступ по приглашению/ссылке и правам.	По умолчанию закрыто: доступ по правам/ссылке (командное пространство).	Публичная библиотека материалов (контент ищется и просматривается на платформе).	Публичный архив: материалы доступны через каталог сайта.
Совместная работа	Совместное редактирование, комментарии, права доступа (в рамках workspace).	Совместное редактирование и комментарии (в рамках пространства).	Коллективность через публикацию/потребление, а не через редактирование одного документа.	Коллективность через публикацию/потребление; совместного редактирования нет.

Продолжение Таблицы 1

Критерий	Notion	Yonote	StuDocu	StudFiles
Избранное	Закладки/избранные страницы (не «избранное материалов» как каталог).	Закладки/быстрый доступ к документам/страницам (логика work-space).	Сохранение/закладки материалов как часть работы с библиотекой.	Закладки/избранное на уровне материалов (если используется), без “каталожной” логики.
Контроль качества / модерация	Контроль через процессы и права команды; нет модерации публичного UGC-каталога.	Контроль через права и регламенты команды; нет витрины UGC для всех.	Проверка/принятие материалов и правила публикации; качество зависит от автора.	Качество зависит от пользователей; возможны дубли и устаревшие версии.
Мотивация публикации	Нет встроенной “награды”; мотивация — удобство команды/личная организация.	Нет встроенной “награды”; мотивация — работа внутри команды.	Награды (Premium-дни) за принятые загрузки.	Обычно без явной награды; мотивация — обмен и доступ к архиву.

1.5 Требования к системе

На основе анализа предметной области, целевой аудитории и обзора аналогов сформулированы функциональные требования к разрабатываемому веб-приложению, а также определён критический путь для каждой пользовательской роли.

1.5.1 Функциональные требования

1. Система должна обеспечивать регистрацию пользователя по уникальному имени пользователя и паролю.
2. Система должна обеспечивать аутентификацию пользователя с хранением JWT в HTTP-only cookie и возможностью выхода из аккаунта.
3. Любой пользователь (в том числе гость) должен иметь возможность просматривать каталог университетов, дисциплин и публикаций без авторизации.
4. Авторизованный пользователь должен иметь возможность создать пост с заголовком, текстом и до пяти изображений в рамках выбранной дисциплины.
5. Авторизованный пользователь должен иметь возможность публиковать посты в разделе «Знания» без привязки к конкретному университету или дисциплине.
6. Автор поста должен иметь возможность редактировать заголовок и текст, а также удалять собственные публикации.
7. Авторизованный пользователь должен иметь возможность добавлять публикации в избранное и убирать их оттуда; список избранного отображается в постоянной боковой панели.
8. Страница поста должна отображать галерею прикрепленных изображений с возможностью открытия каждого в полноэкранном режиме (lightbox).
9. Пользователи с ролью MODERATOR и ADMIN должны иметь возможность удалять любые публикации.
10. Пользователи с ролью ADMIN должны иметь возможность добавлять университеты (с загрузкой логотипа и выбором города) и дисциплины.

1.5.2 Критический путь пользователей

Критический путь описывает минимальную последовательность действий каждой роли для достижения основной цели.

Гость — найти учебный материал:

1. Открыть главную страницу системы.
2. Перейти в каталог университетов, выбрать нужный университет.

3. Выбрать дисциплину из списка.
4. Открыть пост, прочитать текст, просмотреть изображения в галерее.

Студент (USER) — опубликовать материал и сохранить найденный:

1. Зарегистрироваться в системе, затем войти.
2. Перейти к нужной дисциплине, нажать «Добавить пост».
3. Заполнить заголовок, текст, прикрепить изображения, отправить форму.
4. Открыть пост другого пользователя, добавить его в избранное через кнопку-звёздочку.
5. Открыть боковую панель избранного, убедиться, что пост сохранён.
6. Вернуться к своему посту, отредактировать текст или удалить публикацию.

Администратор (ADMIN) — расширить каталог и провести модерацию:

1. Войти в систему под учётной записью администратора.
2. Добавить новый университет: выбрать город, загрузить логотип, указать название.
3. Перейти к созданному университету, добавить дисциплину.
4. Открыть нежелательную публикацию любого пользователя и удалить её.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СИСТЕМЫ

Во второй главе рассматриваются проектные решения и реализация веб-приложения ConSuccess. Первоначально обосновывается выбор технологического стека и приводится описание клиент-серверной архитектуры, отражающее распределение ответственности между фронтендом и бэкендом. Далее представлена модель данных в виде реляционной схемы с описанием каждой сущности и связей между ними, что является основой для корректной работы с базой данных на уровне ORM. В разделе проектирования пользовательского интерфейса описаны макеты ключевых страниц системы с обоснованием принятых компоновочных решений и их влияния на удобство навигации. Завершают главу разделы, посвящённые реализации серверной и клиентской частей приложения с описанием модульной структуры, ключевых компонентов и принципов их взаимодействия.

2.1 Выбор технологического стека

Выбор технологий для разработки веб-приложения ConSuccess осуществлялся исходя из требований к системе, необходимости поддержки серверного рендеринга, удобства работы с базой данных и загрузки файлов, а также возможности обеспечить безопасную авторизацию. Стек разделён на серверную и клиентскую части.

2.1.1 Серверная часть

В качестве серверного фреймворка выбран **NestJS** — фреймворк для Node.js на базе TypeScript. NestJS реализует модульную архитектуру с внедрением зависимостей, предоставляет удобные декораторы для описания маршрутов (@Controller, @Get, @Post), защитников маршрутов (@UseGuards) и перехватчиков файлов (@UseInterceptors). Это позволяет структурировать код по доменным модулям (пользователи, университеты, дисциплины, посты, города) без избыточного дублирования.

Для работы с базой данных применяется **Prisma ORM** — типобезопасный инструмент доступа к данным, обеспечивающий автоматическую генерацию клиента на основе декларативной схемы (schema.prisma). Prisma поддерживает миграции, что упрощает развитие структуры базы данных в процессе разработки. В качестве СУБД используется **PostgreSQL** — реляционная база данных с поддержкой составных первичных ключей (применяется в таблице избранного Favorite), уникальных ограничений и внешних ключей.

Аутентификация реализована на основе **JWT (JSON Web Token)**. При успешном входе сервер формирует токен, подписанный секретным ключом (JWT_SECRET), и возвращает его клиенту. Клиентская сторона хранит токен в HTTP-only cookie, что исключает доступ к нему из JavaScript и снижает риск XSS-атак. Защищённые эндпоинты проверяют токен через AuthGuard, который извлекает Bearer-токен из заголовка Authorization и верифицирует его с помощью JwtService.

Загрузка изображений обеспечивается библиотекой **Multer**, интегрированной в NestJS через FilesInterceptor. Файлы сохраняются на диск в папку uploads/ с именем вида {timestamp}-{random}.{ext}. Принимаются только файлы с MIME-типом image/*; URL сохранённого файла записывается в таблицу Attachment. Папка uploads/ отдаётся клиентам как статические ресурсы по пути /uploads.

2.1.2 Клиентская часть

Клиентская часть разработана на основе **Next.js** (App Router, React 19). Next.js поддерживает серверные компоненты (Server Components), которые получают данные напрямую на сервере

и передают их клиенту в виде готового HTML — это ускоряет первоначальную загрузку и упрощает работу с cookie на сервере. API-маршруты Next.js (app/api/) используются как прокси между клиентом и бэкендом NestJS: они читают cookie access_token, добавляют его в заголовок Authorization: Bearer и перенаправляют запрос на бэкенд.

Для стилизации интерфейса применяется **Tailwind CSS v4** совместно с компонентной библиотекой **shadcn/ui** (стиль new-york, нейтральная цветовая схема). Это обеспечивает единообразный внешний вид всех страниц при минимальном объёме пользовательских стилей. HTTP-запросы к бэкенду выполняются через **Axios** с заранее сконфигурированным экземпляром (baseUrl: http://127.0.0.1:5000).

Управление состоянием форм реализовано с помощью **React Hook Form**: поля проходят клиентскую валидацию, ошибки отображаются под каждым полем, серверные ошибки — через setError("root"). Для кэширования серверных данных и управления повторными запросами используется **TanStack Query v4**.

2.2 Проектирование модели данных

Модель данных системы ConSuccess спроектирована в виде реляционной схемы и описана с помощью Prisma Schema. Схема содержит семь сущностей, связанных отношениями «один ко многим» и «многие ко многим» (через таблицу-связку).

User — пользователь системы. Хранит уникальное имя пользователя (username), хеш пароля и роль (USER, MODERATOR, ADMIN). Роль определяет права: обычный пользователь создаёт и редактирует собственные публикации; модератор и администратор могут удалять любые публикации; администратор дополнительно управляет каталогом (добавляет университеты и дисциплины).

City — город. Содержит уникальное название. Используется при создании университета для указания местоположения.

University — университет. Содержит название, ссылку на город (cityId) и необязательный путь к логотипу (avatar). Является корневым уровнем иерархии каталога.

Subject — учебная дисциплина. Привязана к конкретному университету через внешний ключ universityId. Образует второй уровень иерархии каталога.

Post — публикация. Содержит заголовок (title), основной текст (body), дату создания (createdAt), идентификатор автора (userId) и необязательный идентификатор дисциплины (subjectId). Если subjectId равен null, публикация относится к разделу «Знания» и не привязана к конкретному университету или дисциплине.

Attachment — вложение (изображение). Хранит относительный URL файла на сервере (вида /uploads/имя-файла.расширение) и идентификатор поста (postId). Один пост может иметь до пяти вложений.

Favorite — избранное. Таблица-связка между пользователем и публикацией. Имеет составной первичный ключ [userId, postId], что исключает дублирование записей при повторном добавлении одного и того же поста в избранное.

Таблица 5 – Сущности модели данных

Сущность	Ключевые поля	Связи
User	id, username (unique), passwordHash, role	→ Post (один ко многим), → Favorite (один ко многим)
City	id, name (unique)	→ University (один ко многим)
University	id, name, cityId, avatar?	← City, → Subject (один ко многим)
Subject	id, name, universityId	← University, → Post (один ко многим)
Post	id, title, body, subjectId?, userId, createdAt	← Subject (необязательно), ← User, → Attachment, → Favorite

Продолжение Таблицы 5

Сущность	Ключевые поля	Связи
Attachment	id, url, postId	← Post
Favorite	userId + postId (составной РК)	← User, ← Post

2.3 Проектирование макетов пользовательского интерфейса

На этапе проектирования для каждой ключевой страницы системы были разработаны макеты пользовательского интерфейса. Макеты отражают структуру страниц, расположение элементов навигации, контентных блоков и интерактивных элементов управления. В качестве основы принята единая компоновочная схема: горизонтальная навигационная панель в верхней части страницы, фиксированная боковая панель избранного слева и основная контентная область, занимающая оставшееся пространство. Такая структура обеспечивает единообразие восприятия и упрощает ориентацию пользователя в системе.

Ниже представлено описание макетов основных страниц приложения ConSuccess.

2.3.1 Главная страница

Главная страница является точкой входа в систему. В верхней части расположена навигационная панель с логотипом, ссылками на разделы (Главная, Вузы, Знания, О продукте) и кнопкой входа. Слева находится панель избранного, в которой авторизованным пользователям отображаются сохранённые публикации. Основная часть страницы содержит блок «Последние добавленные материалы» — горизонтальный ряд карточек публикаций. Каждая карточка включает метку раздела (название вуза и дисциплины либо «Знания»), заголовок, краткий фрагмент текста, дату публикации и ссылку «Открыть →». Такое оформление позволяет пользователю быстро оценить актуальные материалы сразу при открытии приложения.

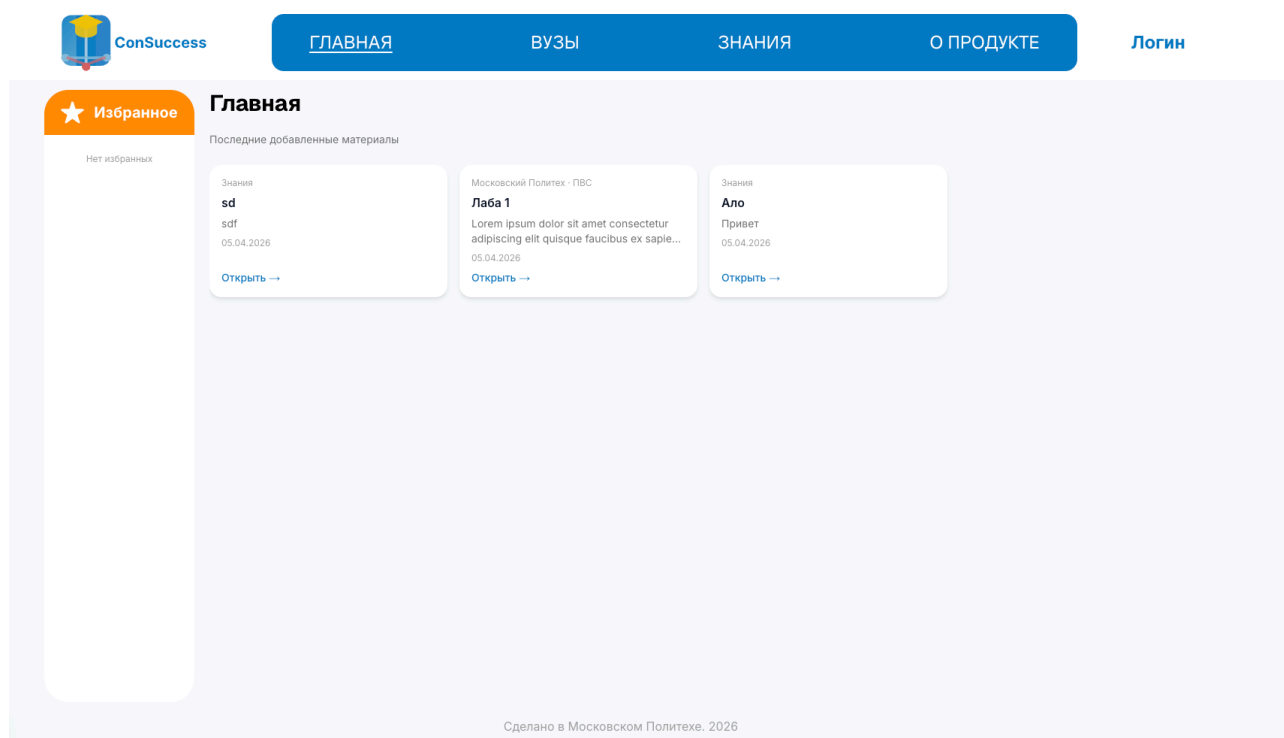


Рисунок 7: Макет главной страницы

2.3.2 Страница «О продукте»

Страница «О продукте» содержит описание платформы и предназначена для ознакомления новых пользователей с её возможностями. Страница разделена на несколько смысловых блоков: краткое определение сервиса ConSuccess, пошаговое описание принципа работы (выбор вуза → дисциплина → публикация материала), раздел «Возможности» с карточками четырёх основных функций (материалы по предметам, раздел «Знания», избранное, поиск по вузам) и раздел «Для кого», адресованный студентам, которые хотят обмениваться материалами. Страница сохраняет общую компоновку с навигационной панелью и боковой панелью избранного.

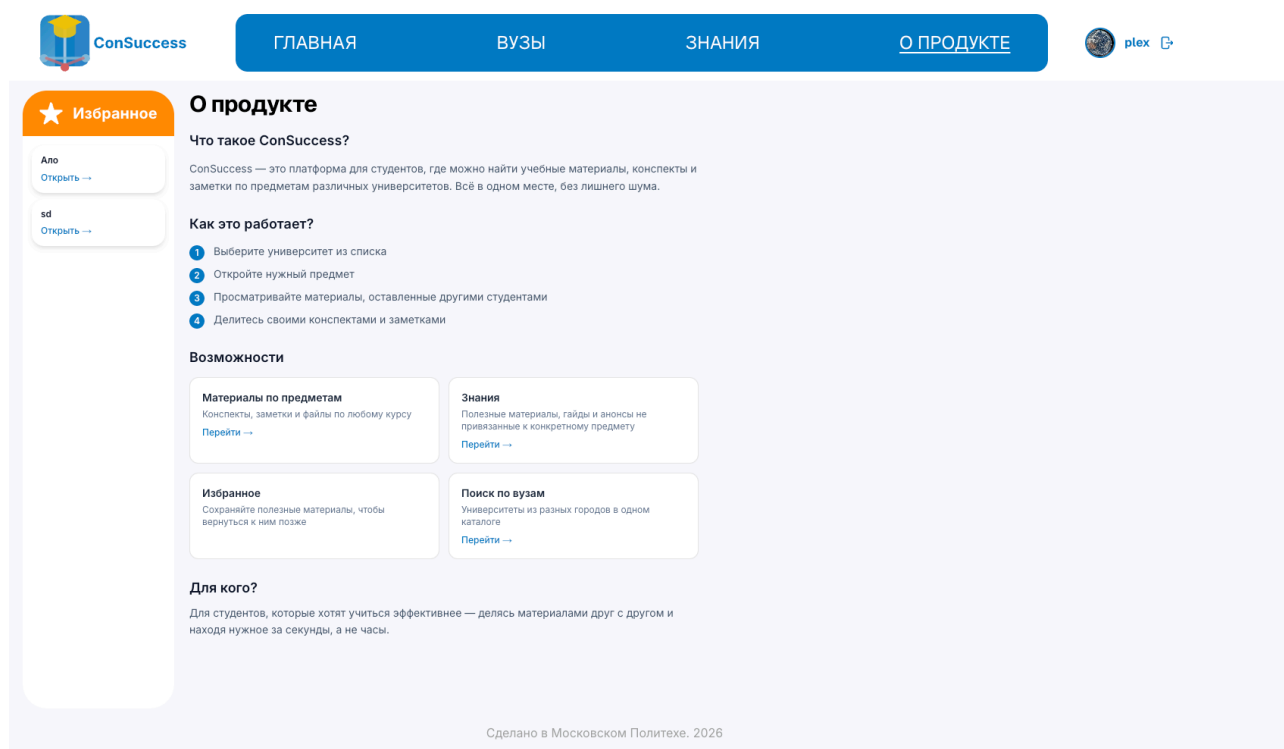


Рисунок 8: Макет страницы «О продукте»

2.3.3 Страница каталога вузов

Страница каталога вузов отображает все зарегистрированные в системе учебные заведения. Карточки вузов расположены в адаптивной сетке; каждая карточка содержит логотип (или заглушку «Лого»), название вуза, город и ссылку «Открыть материалы →». Для авторизованных пользователей с ролью администратора в верхней части страницы доступна кнопка «Добавить», открывающая форму создания нового вуза. Боковая панель избранного в данном разделе отображает последние сохранённые публикации пользователя, обеспечивая быстрый доступ к ним без перехода на другие страницы.

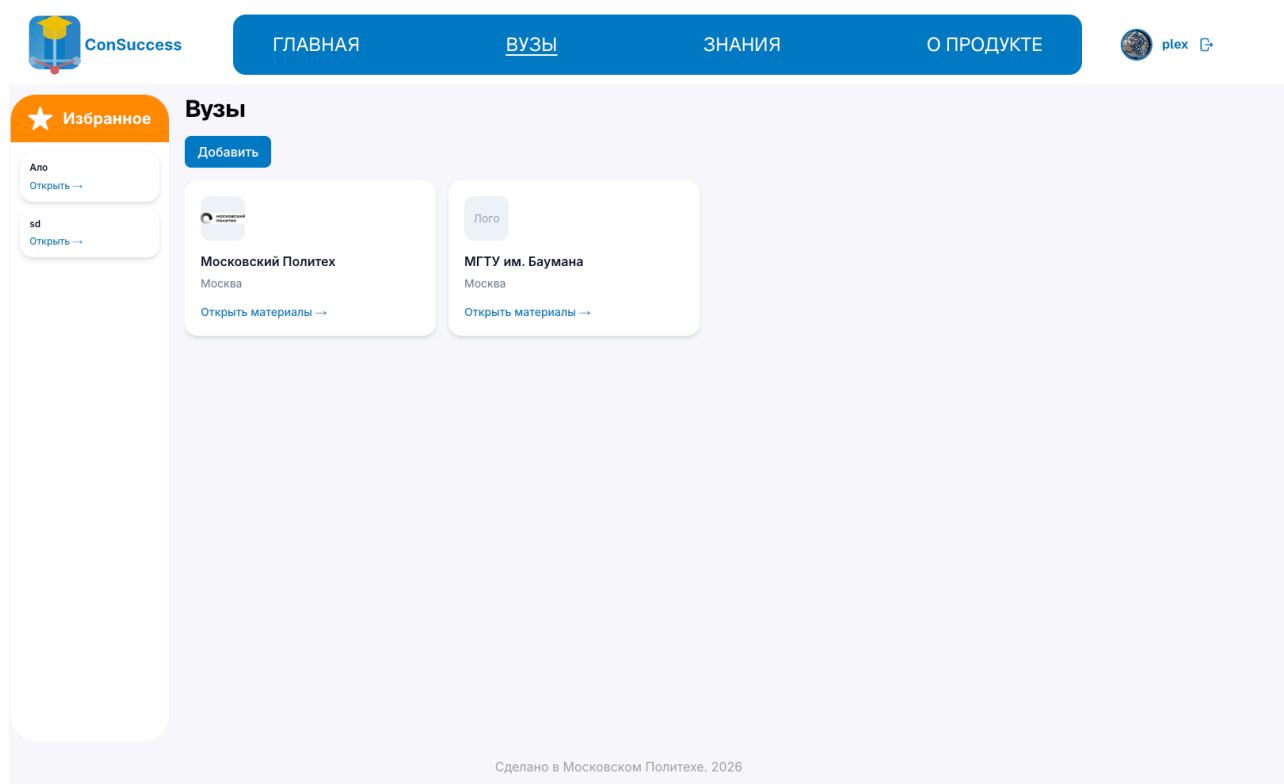


Рисунок 9: Макет страницы каталога вузов

2.3.4 Страница предметов вуза

При переходе в карточку вуза открывается страница с перечнем его дисциплин. В заголовке указывается название вуза, город и ссылка «← Назад» для возврата к каталогу. Дисциплины представлены карточками с иконкой папки, названием предмета и ссылкой «Открыть материалы →». Кнопка «Добавить» доступна пользователям с правами администратора. Пустое состояние (нет дисциплин) не показывает сетку, но сохраняет заголовок и кнопку добавления, что сигнализирует о возможности наполнения раздела.

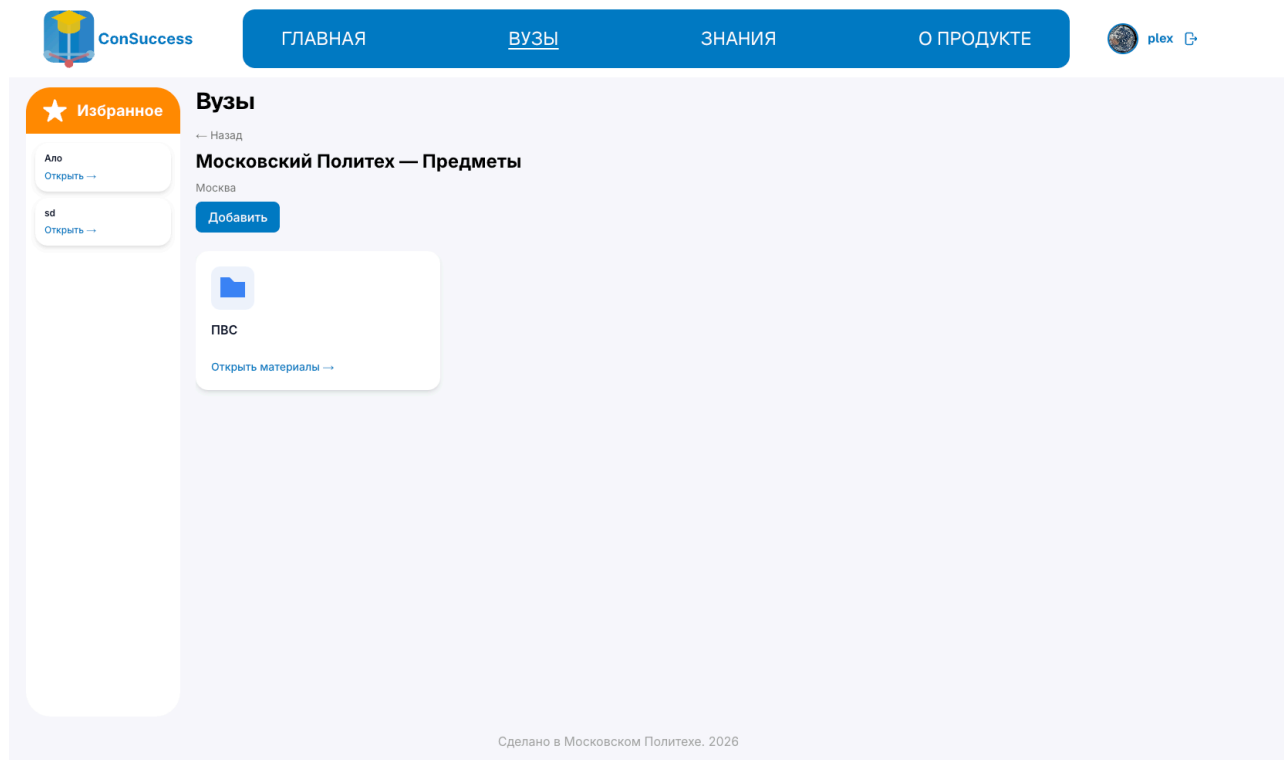


Рисунок 10: Макет страницы предметов вуза

2.3.5 Страница публикаций по предмету

Страница публикаций открывается после выбора дисциплины и отображает все материалы, привязанные к данному предмету. В заголовке указывается название вуза и дисциплины. Публикации представлены карточками, каждая из которых содержит миниатюру первого прикрепленного изображения (при наличии), заголовок, краткий фрагмент текста и ссылку «Открыть →». Авторизованным пользователям доступна кнопка «Добавить» для перехода к форме создания новой публикации. Единообразие карточек облегчает визуальное сканирование списка.

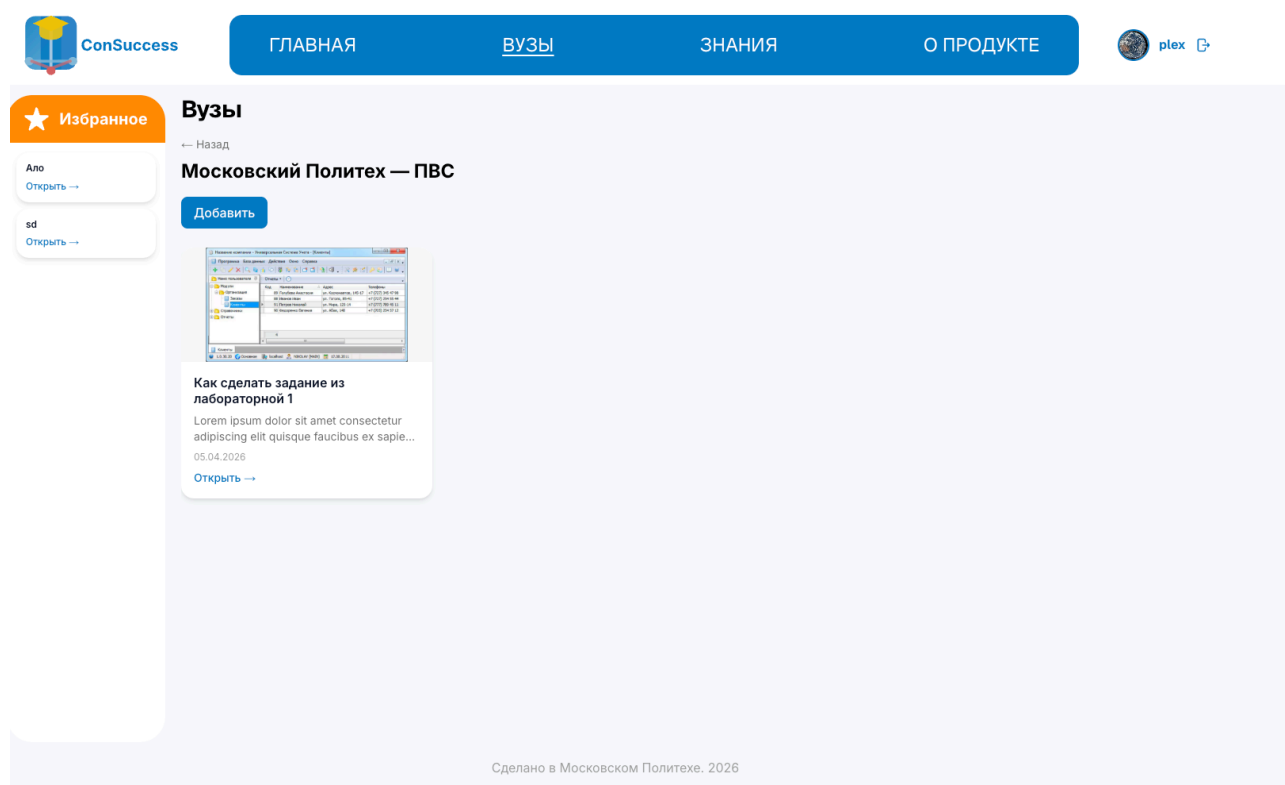


Рисунок 11: Макет страницы публикаций по предмету

2.3.6 Страница просмотра публикации

Страница публикации отображает полное содержимое материала. В верхней части размещена ссылка «← Назад» для возврата к списку публикаций. Далее следуют заголовок поста, имя автора и дата публикации. Основную часть занимает полный текст материала. Ниже текста располагается галерея прикрепленных изображений; каждое изображение можно открыть в полноэкранном режиме (lightbox). Для авторизованных пользователей доступна кнопка «В избранное» (звёздочка) для сохранения публикации. Автор поста и пользователи с ролью модератора или администратора видят дополнительные кнопки «Редактировать» и «Удалить».

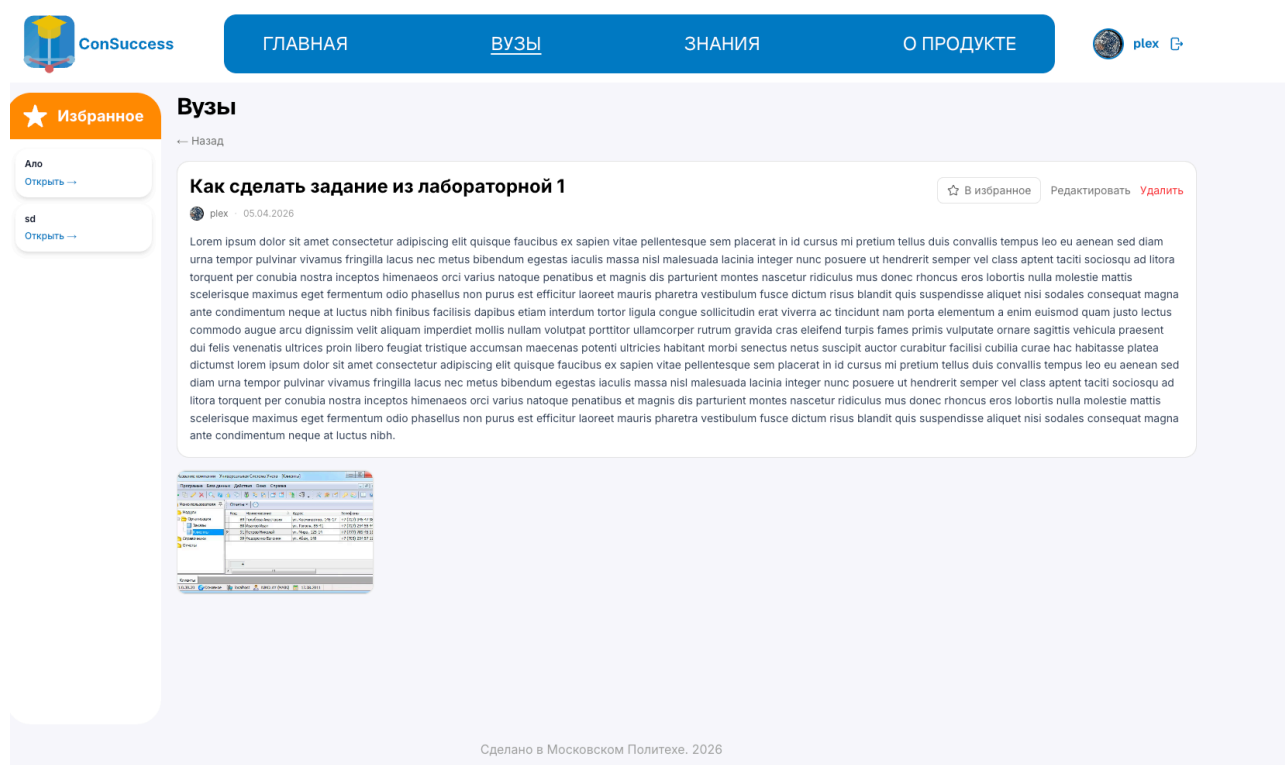


Рисунок 12: Макет страницы просмотра публикации

2.4 Реализация серверной части

Серверная часть веб-приложения ConSuccess реализована на фреймворке NestJS с использованием TypeScript. Выбор NestJS обусловлен его модульной архитектурой с поддержкой внедрения зависимостей (Dependency Injection), наличием готовых декораторов для описания маршрутов и упрощённой интеграцией с библиотеками обработки HTTP-запросов, файлов и JWT. Серверная часть предоставляет набор REST-эндпоинтов, к которым клиент обращается для получения и изменения данных каталога, работы с публикациями, авторизации и загрузки изображений.

2.4.1 Структура серверного приложения

Серверное приложение построено вокруг корневого модуля AppModule, в котором регистрируются все контроллеры и сервисы. В рамках проекта принята плоская организация: вместо создания отдельного модуля для каждой доменной области все контроллеры и сервисы подключаются непосредственно в корневом модуле. Это оправдано относительно небольшим количеством доменных областей (пользователи, университеты, дисциплины, города, публикации, избранное, вложения) и упрощает навигацию по кодовой базе.

Корневой модуль импортирует:

1. ConfigModule — для чтения переменных окружения (строки подключения к базе данных и секрета подписи JWT) из файла .env.
2. PrismaModule — глобальный модуль, предоставляющий синглтон PrismaService для доступа к базе данных из любых сервисов.
3. AuthModule — модуль авторизации, инкапсулирующий AuthService, AuthController и JwtModule с настройкой секрета из ConfigService.

Точка входа приложения (main.ts) создаёт экземпляр NestExpressApplication, настраивает CORS для запросов с фронтенда (http://localhost:3000), создаёт каталог uploads/ (при его отсутствии) и регистрирует его как источник статических ресурсов по пути /uploads. Это позволяет клиенту запрашивать загруженные изображения напрямую у серверной части без дополнительного проксирования.

2.4.2 Доменные сервисы и контроллеры

Для каждой сущности каталога в проекте реализованы пары «контроллер — сервис». Контроллеры отвечают за приём HTTP-запросов, извлечение параметров маршрута и тела запроса, применение защитников (AuthGuard) и перехватчиков файлов (FilesInterceptor). Сервисы инкапсулируют бизнес-логику и работу с базой данных через PrismaService.

1. `UserController` и `UserService` — управляют пользователями: создание, получение по идентификатору, получение по имени пользователя (используется в процессе авторизации). Пароли пользователей хранятся в виде bcrypt-хеша с десятью раундами соления; функции хеширования вынесены в модуль `utils/crypt.ts`.
2. `UniversitiesController` и `UniversitiesService` — предоставляют операции получения списка университетов и отдельного университета по идентификатору, а также создание нового университета (с загрузкой логотипа и указанием города). При создании университета контроллер использует `FileInterceptor` для приёма файла логотипа и сохраняет его в каталог `uploads/`.
3. `CitiesController` и `CitiesService` — отвечают за получение и создание городов, используемых при регистрации новых университетов.
4. `SubjectsController` и `SubjectsService` — реализуют CRUD-операции для дисциплин в контексте выбранного университета. Все операции принимают `universityId` как параметр маршрута, что обеспечивает иерархическую изоляцию дисциплин по университетам.
5. `PostsController` и `PostsService` — управляют публикациями в рамках выбранной дисциплины, а также публикациями раздела «Знания». Эндпоинт создания поста защищён `AuthGuard`, использует `FilesInterceptor` для приёма до пяти изображений и сохраняет их в каталог `uploads/`. Сервис создаёт запись `Post` и набор записей `Attachment`, связанных с постом.
6. `FavoritesController` и `FavoritesService` — реализуют добавление и удаление публикаций из избранного для текущего авторизованного пользователя, а также получение списка избранных постов. Идентификатор пользователя извлекается из JWT-токена.

2.4.3 Модуль авторизации

Модуль авторизации инкапсулирует логику входа пользователя и проверку доступа к защищённым эндпоинтам.

`AuthService` принимает имя пользователя и пароль, запрашивает пользователя через `UserService.getUserByName`, сравнивает переданный пароль с сохранённым хешем через `bcrypt`. В случае успеха формируется JWT-токен с полезной нагрузкой `{ sub: userId, username, role }` и подписью секретом `JWT_SECRET`. Токен возвращается клиенту.

`AuthController` предоставляет два эндпоинта: `POST /auth/login` для получения токена и `GET /auth/me`, защищённый `AuthGuard`. Последний возвращает данные текущего пользователя, что позволяет клиенту проверить валидность сессии и получить актуальные сведения о роли.

AuthGuard реализует интерфейс CanActivate. Guard извлекает Bearer-токен из заголовка Authorization, проверяет его подпись через JwtService.verifyAsync и при успехе прикрепляет раскодированную полезную нагрузку к объекту запроса (request.user). Это делает данные пользователя доступными во всех контроллерах, использующих @UseGuards(AuthGuard). При отсутствии или недействительности токена guard возбуждает UnauthorizedException, что приводит к ответу клиенту со статусом 401.

2.4.4 Работа с базой данных через Prisma

Для доступа к базе данных используется Prisma ORM. Декларативная схема базы данных описана в файле schema.prisma и содержит модели User, City, University, Subject, Post, Attachment, Favorite. Для перечислимых значений роли пользователя определён enum Role со значениями USER, MODERATOR и ADMIN.

Все изменения структуры базы данных выполняются через миграции Prisma (npx prisma migrate dev). Миграции хранятся в каталоге prisma/migrations/ под именами, отражающими дату и семантику изменения: добавление ролей, добавление поля аватара, связь с городом, добавление поля видимости поста. Такой подход обеспечивает воспроизводимость среды и упрощает развёртывание на новых серверах.

PrismaService расширяет PrismaClient и переопределяет onModuleInit и onModuleDestroy для корректного управления подключением к базе данных в рамках жизненного цикла NestJS. Сервис зарегистрирован в глобальном модуле PrismaModule, что делает его доступным для всех сервисов без повторного импорта.

Запросы к базе данных формируются через типобезопасный Prisma Client. Например, при получении публикации вместе с вложениями используется включение связанной сущности (include: { attachments: true }), а при фильтрации избранного — составной where с идентификатором пользователя.

2.4.5 Загрузка и хранение изображений

Для загрузки изображений используется библиотека Multer, интегрированная в NestJS через декоратор FilesInterceptor. Приёмник файлов настраивается через MulterOptions:

1. storage — используется diskStorage для сохранения файлов в каталог uploads/ на локальной файловой системе.
2. filename — генерация имени файла по шаблону {timestamp}-{random}.{ext}, где timestamp — текущая метка времени в миллисекундах, random — случайное шестизначное

число, `ext` — оригинальное расширение файла. Такая схема исключает коллизии имён при одновременных загрузках.

3. `fileFilter` — проверка MIME-типа: принимаются только файлы, начинающиеся с `image/`. Файлы с другими типами отклоняются.

4. `limits` — ограничение максимального размера файла и количества файлов в одном запросе.

После успешного сохранения файла `multer` добавляет его метаданные в массив `files` запроса. Сервис публикаций перебирает массив и создаёт запись `Attachment` для каждого файла: в поле `url` записывается относительный путь `/uploads/имя-файла.расширение`, который клиент может дополнять базовым URL сервера для получения файла через статический маршрут.

2.4.6 Обработка ошибок и валидация

Для обработки ошибок используются встроенные исключения NestJS: `UnauthorizedException` (401), `ForbiddenException` (403), `NotFoundException` (404), `BadRequestException` (400). Исключения автоматически преобразуются в HTTP-ответы с соответствующими статусами и сообщениями.

Валидация входных данных выполняется на уровне контроллеров. При отсутствии обязательных полей или неверном формате данных возбуждается `BadRequestException`. Для защиты от SQL-инъекций используется параметризация запросов Prisma: все значения передаются в виде аргументов, а не конкатенируются в строку запроса, что исключает возможность внедрения постороннего SQL-кода.

Таким образом, серверная часть приложения обеспечивает модульную структуру, безопасную авторизацию через JWT, надёжную работу с базой данных через Prisma, корректную обработку загружаемых изображений и предсказуемую обработку ошибок. Данные архитектурные и реализационные решения формируют устойчивую основу для расширения функциональности и поддержки приложения в дальнейшем.

2.5 Реализация клиентской части

Клиентская часть веб-приложения ConSuccess реализована на фреймворке Next.js (модель App Router) с использованием React 19. Next.js сочетает преимущества серверного и клиентского рендеринга: серверные компоненты получают данные непосредственно на сервере и отправляют клиенту уже сформированный HTML, а клиентские компоненты обеспечивают интерактивность (формы, галереи, работа с состоянием). Такой подход снижает первоначальное время загрузки страниц и упрощает управление секретами авторизации, поскольку JWT-токен не передаётся в JavaScript-код браузера.

2.5.1 Структура клиентского приложения

Клиентское приложение построено в соответствии с соглашениями App Router. Основные каталоги:

1. `app/` — корневой каталог маршрутизации. Содержит разметку приложения (`layout.tsx`), глобальные стили и метаданные.
2. `app/(user)/` — группа маршрутов для авторизованной зоны, оформленная как скобочная группа Next.js. Это позволяет объединять страницы общим макетом без влияния на URL. Общий макет содержит горизонтальную навигационную шапку, а также компонент `FavoritesSidebar`, в котором авторизованным пользователям отображаются сохранённые публикации на каждой странице зоны.
3. `app/api/` — серверные обработчики маршрутов (`route.ts`), играющие роль прокси между клиентом и бэкендом. Вся логика работы с `cookie access_token` вынесена сюда.
4. `app/_components/` — переиспользуемые компоненты интерфейса: `Header`, `Navlink`, `HeaderProfile`, `UserActions` (выход из аккаунта, пользовательское меню), `PostCard`, `KnowledgePostCard`, `FavoriteButton` (переключатель избранного в виде звёздочки), `ImageGallery` (компонент галереи с лайтбоксом).
5. `app/_icons/` — централизованный реестр иконок. SVG-файлы импортируются из каталога `assets/` и реэкспортируются по именам (`ConSuccessLogo`, `Star`, `LogoutIcon`, `FolderIcon`), что упрощает подключение иконок в любых компонентах и сокращает дублирование импортов.
6. `shared/api/` — клиентский слой работы с API: базовый экземпляр `Axios` (`config/`) с `baseURL: process.env.API_URL ?? 'http://127.0.0.1:5000'`, строковые константы эндпоинтов (`endpoints/`), функции доступа к конкретным ресурсам (`auth/`, `universities/`, `subjects/`, `posts/`).

7. `shared/types/` — TypeScript-типы сущностей домена (`City`, `University`, `Subject`, `Post`, `Attachment`).
8. `lib/` — вспомогательные функции; в частности, утилита `cn` для объединения имён классов Tailwind на основе `clsx` и `tailwind-merge`.

Помимо основной зоны каталога, в приложении реализованы страницы `/login` и `/register` (последняя — с подтверждением пароля), а также отдельный раздел `/knowledge` для публикаций, не привязанных к конкретной дисциплине, со страницами `/knowledge/add-post` и `/knowledge/posts/[postId]`.

2.5.2 Страницы и серверные компоненты

Большинство страниц приложения являются серверными компонентами. Такие компоненты имеют доступ к `cookies()` и могут выполнять запросы к API напрямую на сервере. Это позволяет выполнять условный рендеринг на основе признака авторизации: например, кнопки «Добавить» на страницах каталога университетов и дисциплин отображаются только пользователям с ролью администратора, а кнопки редактирования и удаления поста — только автору публикации или модератору. Правила владения постом проверяются по совпадению `userId` текущего пользователя с автором поста.

Для страниц, требующих интерактивности (формы публикации, галерея с лайтбоксом, переключатель избранного), создаются отдельные клиентские компоненты в каталогах `_components/` рядом с соответствующей страницей. Сервер формирует необходимые данные (например, название университета и дисциплины для контекста формы) и передаёт их в виде `props` клиентскому компоненту. Разделение ответственности между серверным и клиентским компонентом улучшает производительность и обеспечивает корректную работу со стейтом формы.

Страницы с параметрами маршрута (`[id]`, `[subjectId]`, `[postId]`) используют асинхронные функции серверных компонентов и возвращают разметку на основе полученных данных. При отсутствии запрашиваемой сущности вызывается `notFound()`, что приводит к отображению страницы 404.

Несколько параллельных запросов данных на одной странице выполняются через `Promise.all`, что уменьшает общее время ожидания. Например, на странице университета одновременно запрашиваются сведения об университете и список его дисциплин. Страницы, использующие `cookies()`, становятся автоматически динамическими, поэтому для них не применяется `generateStaticParams`.

2.5.3 Серверные маршруты в роли прокси

Next.js API-маршруты (`app/api/.../route.ts`) используются для проксирования запросов от клиента к бэкенду NestJS. Данный подход решает несколько задач:

1. Скрытие базового URL бэкенда от клиентского JavaScript-кода и упрощение настройки окружения.
2. Централизованное управление cookie `access_token`: в одном месте выполняется чтение токена, добавление его в заголовок `Authorization: Bearer ...` и перенаправление запроса.
3. Поддержка HTTP-only cookie: клиентский JavaScript не имеет доступа к токenu, что снижает риск XSS-атак; всё взаимодействие с токеном происходит на стороне сервера Next.js.

Все прокси-маршруты построены по единой схеме. Сначала читается `access_token` из cookie; при отсутствии возвращается статус 401. Затем формируется запрос через экземпляр `axiosApi` к соответствующему эндпоинту бэкенда с заголовком `Authorization: Bearer <token>`. Для JSON-запросов тело читается через `await req.json()` и передаётся как `body`. Для запросов с загрузкой файлов (`POST /subjects/:subjectId/posts`, `POST /universities`, `POST /universities/:id/subjects`) тело передаётся в виде `Buffer.from(await req.arrayBuffer())` вместе с исходным заголовком `Content-Type`, что сохраняет границу `multipart` и обеспечивает корректную обработку нескольких файлов на бэкенде. Повторное парсинг `FormData` в прокси не выполняется — это важное требование, поскольку любое чтение и повторная сборка тела запроса нарушает `multipart`-границу и приводит к потере файлов. При ошибке `Axios` (`axios.isAxiosError`) статус и тело ответа бэкенда пробрасываются клиенту без изменений; при других исключениях возвращается ответ со статусом 500.

Отдельно стоит отметить маршрут авторизации `POST /api/auth/login`: он проксирует запрос на бэкенд, а при успешном ответе сохраняет полученный JWT-токен в HTTP-only cookie. Этот механизм полностью инкапсулирует работу с токеном — клиентский код не видит и не сохраняет токен самостоятельно. Дополнительно маршрут `GET /api/auth/me` используется защищёнными страницами для проверки валидности сессии и получения актуальных сведений о роли пользователя.

2.5.4 Формы и работа с данными

Формы (регистрация, вход, создание университета, создание дисциплины, создание и редактирование публикации) реализованы с использованием библиотеки `React Hook Form`. Она предоставляет удобный API для регистрации полей, отслеживания состояния валидации и отправки формы.

Ключевые соглашения форм:

1. Ошибки валидации полей отображаются непосредственно под соответствующим полем.
2. Ошибки сервера сохраняются через `setError("root", { message })` и отображаются над полями формы. Это отделяет сообщения об ошибках валидации от ошибок уровня запроса и упрощает единообразное отображение.
3. Состояние «форма отправляется» отслеживается через `formState.isSubmitting` и используется для блокировки кнопки отправки, что предотвращает повторные отправки.
4. Над заголовком формы размещается ссылка «← Назад» для быстрого возврата к предыдущей странице.

Для управления серверным состоянием и кэширования используется библиотека `TanStack Query`. Она позволяет декларативно описывать запросы данных, автоматически обрабатывать их жизненный цикл (загрузка, успех, ошибка) и инвалидировать кэш при изменении связанных ресурсов. Всплывающие уведомления об успехе или ошибке операций реализованы через библиотеку `Sonner`, что обеспечивает единообразный и неблокирующий пользовательский опыт.

2.5.5 Отображение публикаций и загрузка изображений

Публикации отображаются в виде карточек (`PostCard` для постов дисциплины и `KnowledgePostCard` для раздела «Знания»). Каждая карточка содержит превью первого прикрепленного изображения, заголовок, краткий фрагмент текста (2 строки) и ссылку «Открыть →» на полную страницу поста. При отображении изображений соблюдаются следующие правила:

1. Изображения, отдаваемые бэкендом по пути `/uploads/...`, префиксуются базовым URL сервера (`apiURL`) из модуля `shared/api/config`.
2. Компонент `next/image` используется с параметром `unoptimized: true`, поскольку оптимизатор `Next.js` по умолчанию отклоняет запросы к приватным IP-адресам.
3. Для отображения полного изображения без обрезки используется `object-contain` вместо `object-cover`. Это важно для учебных материалов, где часть содержимого (например, текст на отсканированной странице) не должна обрезаться. Фон карточки задаётся классом `bg-neutral-50`, что обеспечивает единообразный вид при разных соотношениях сторон изображений.
4. В файле `next.config.ts` в секции `images.remotePatterns` разрешены хосты `localhost:5000` и `127.0.0.1:5000`, используемые при локальной разработке.

Для просмотра изображения в полноэкранном режиме реализован клиентский компонент `ImageGallery`, открывающий изображение в лайтбоксе по клику. Компонент управляет состоянием открытой карточки, блокирует прокрутку страницы при открытом изображении и закрывает лайтбокс по клику вне изображения или по нажатию клавиши `Escape`.

Переключатель избранного реализован как отдельный клиентский компонент `FavoriteButton` в форме звёздочки. При нажатии он отправляет запрос на добавление или удаление публикации из избранного и обновляет соответствующее состояние. Список избранного формируется в боковой панели `FavoritesSidebar`, доступной на всех страницах авторизованной зоны.

2.5.6 Стилизация интерфейса

Для стилизации используется `Tailwind CSS` версии 4 совместно с библиотекой `shadcn/ui` (стиль `new-york`, нейтральная палитра). Базовые компоненты (кнопки, поля ввода, карточки, тосты) импортируются из `components/ui/`. Собственные компоненты строятся поверх них, что обеспечивает единообразный визуальный стиль и минимизирует количество пользовательских стилей. Иконки берутся из библиотеки `Lucide`.

Компоновка страниц приложения использует `flex`-контейнеры: боковая панель слева (избранное и навигация) и основная контентная область с классами `flex-1 min-w-0`. Модификатор `min-w-0` необходим для корректной работы переноса длинных строк внутри контентной области — без него контент может быть принудительно ужат до ширины текста, что приводит к нежелательным переносам и горизонтальной прокрутке страницы. Для переноса длинных непрерывных строк в `Tailwind CSS v4` используется утилита `wrap-break-word`.

Таким образом, клиентская часть приложения обеспечивает удобную иерархическую навигацию по каталогу, безопасную работу с токенами через серверные API-маршруты, качественное отображение изображений с поддержкой лайтбокса, а также единообразный и адаптивный пользовательский интерфейс. Разделение ответственности между серверными и клиентскими компонентами позволяет использовать сильные стороны обоих подходов и обеспечивает высокую производительность приложения.

3 ТЕСТИРОВАНИЕ И ОЦЕНКА СИСТЕМЫ

Третья глава посвящена вопросам оценки качества разработанной системы. В ней рассматриваются сценарии функционального тестирования для трёх пользовательских ролей (гость, студент, администратор), меры по обеспечению информационной безопасности и возможности масштабирования и дальнейшего развития платформы `ConSuccess`. Целью

раздела является подтверждение соответствия системы функциональным требованиям, сформулированным в первой главе, анализ потенциальных угроз и описание перспективных направлений развития.

Сценарий функционального тестирования составлен на основе критического пути пользователей и охватывает три ключевые роли: Гость, Студент (USER) и Администратор (ADMIN). Для каждой роли определены тестируемые шаги, ожидаемый результат и критерий прохождения.

3.1 Сценарий тестирования для роли «Гость» — поиск учебного материала

Цель: убедиться, что неавторизованный пользователь может просматривать каталог и публикации без входа в систему.

Таблица 2 – Сценарий тестирования роли «Гость»

№	Шаг	Ожидаемый результат	Статус
1	Открыть главную страницу системы без авторизации.	Отображается список последних публикаций. Кнопка «Логин» видна в навигации.	

Продолжение Таблицы 2

№	Шаг	Ожидаемый результат	Статус
2	Перейти в раздел «Вузы».	Отображается сетка карточек университетов с логотипами, названиями и городами.	
3	Выбрать университет из списка.	Открывается страница дисциплин выбранного вуза. Присутствует ссылка «← Назад».	
4	Выбрать дисциплину из списка.	Отображаются карточки публикаций с превью изображений и фрагментом текста.	
5	Открыть публикацию по ссылке «Открыть →».	Отображается полный текст поста, данные об авторе, дата, галерея изображений. Кнопки «Редактировать» и «Удалить» не отображаются.	
6	Нажать на изображение в галерее.	Изображение открывается в полноэкранном режиме (lightbox).	

3.2 Сценарий тестирования для роли «Студент (USER)» — публикация материала и работа с избранным

Цель: убедиться, что авторизованный пользователь может создавать, редактировать и удалять публикации, а также пользоваться механизмом избранного.

Таблица 3 – Сценарий тестирования роли «Студент (USER)»

№	Шаг	Ожидаемый результат	Статус
1	Зарегистрироваться в системе с уникальным именем пользователя и паролем.	Учётная запись создана. Выполнен автоматический вход или предложена страница входа.	
2	Войти в систему.	В навигации отображается имя пользователя и кнопка выхода. Боковая панель «Избранное» доступна.	
3	Перейти к нужной дисциплине и нажать «Добавить».	Открывается форма создания публикации с полями «Заголовок», «Текст», «Изображения».	
4	Заполнить заголовок и текст, прикрепить одно изображение, нажать «Добавить».	Публикация создана. Выполнено перенаправление на страницу дисциплины. Новый пост отображается в списке.	
5	Открыть публикацию другого пользователя и нажать кнопку-звёздочку «В избранное».	Пост добавлен в избранное. В боковой панели появляется ссылка на сохранённую публикацию.	
6	Повторно нажать кнопку-звёздочку на том же посте.	Пост удалён из избранного. Ссылка исчезает из боковой панели.	
7	Открыть собственную публикацию и нажать «Редактировать».	Открывается форма редактирования с текущими данными. После сохранения изменения отображаются на странице поста.	
8	Открыть собственную публикацию и нажать «Удалить».	Публикация удалена. Выполнено перенаправление на страницу дисциплины. Пост отсутствует в списке.	

3.3 Сценарий тестирования для роли «Администратор (ADMIN)» — управление каталогом и модерация

Цель: убедиться, что администратор может добавлять университеты и дисциплины, а также удалять публикации любых пользователей.

Таблица 4 – Сценарий тестирования роли «Администратор (ADMIN)»

№	Шаг	Ожидаемый результат	Статус
1	Войти в систему под учётной записью с ролью ADMIN.	Выполнен вход. На страницах каталога отображается кнопка «Добавить» для университетов и дисциплин.	
2	Перейти в раздел «Вузы» и нажать «Добавить». Заполнить название, выбрать город, загрузить логотип.	Университет создан и отображается в каталоге с загруженным логотипом и городом.	
3	Перейти на страницу созданного университета и нажать «Добавить» для создания дисциплины.	Дисциплина создана и отображается в списке предметов университета.	
4	Открыть публикацию любого пользователя.	На странице поста отображаются кнопки «Редактировать» и «Удалить» вне зависимости от авторства.	
5	Нажать «Удалить» на публикации другого пользователя.	Публикация удалена. Выполнено перенаправление на страницу дисциплины. Пост отсутствует в списке.	

3.4 Информационная безопасность

Вопросы информационной безопасности имеют ключевое значение для веб-приложений, обрабатывающих пользовательские учётные записи и загружаемый контент. В системе Con-Success реализован набор мер, направленных на защиту учётных данных пользователей, предотвращение несанкционированного доступа к защищённым операциям и противодействие распространённым уязвимостям веб-приложений. В данном разделе описаны принятые проектные решения и особенности их реализации.

3.4.1 Хранение паролей

Пароли пользователей никогда не сохраняются в базе данных в открытом виде. При регистрации пользователя пароль хешируется алгоритмом `bcrypt` с коэффициентом десяти раундов соления. Функции хеширования и сравнения вынесены в модуль `utils/crypt.ts` серверной части. При входе в систему введённый пароль сравнивается с сохранённым хешем через функцию `bcrypt`, что исключает возможность восстановления исходного пароля даже при утечке базы данных. Выбор `bcrypt` обусловлен его устойчивостью к перебору: алгоритм намеренно спроектирован медленным и адаптивным, что замедляет атаки перебором паролей и атаки по словарю.

3.4.2 Авторизация через JWT

Авторизация пользователей реализована на базе JSON Web Token (JWT). При успешной проверке учётных данных `AuthService` формирует токен, подписанный секретным ключом `JWT_SECRET`, хранящимся в переменных окружения. Полезная нагрузка токена содержит идентификатор пользователя, имя пользователя и роль, что позволяет серверу восстанавливать контекст пользователя без обращения к базе данных при каждом запросе.

Защищённые эндпоинты используют `AuthGuard`, который выполняет следующие действия:

1. Извлекает Bearer-токен из заголовка `Authorization`.
2. Проверяет подпись токена через `JwtService.verifyAsync` с использованием секрета из `ConfigService`.
3. При успехе прикрепляет раскодированную полезную нагрузку к объекту запроса (`request.user`), делая данные пользователя доступными в контроллерах.
4. При отсутствии или недействительности токена возбуждает `UnauthorizedException`, что приводит к ответу со статусом 401.

Подпись токена защищает его от подделки: любой изменённый токен не пройдёт проверку подписи. Срок действия токена задаётся при его генерации и ограничивает время, в течение которого украденный токен может быть использован.

3.4.3 HTTP-only cookies

Для хранения JWT-токена на стороне клиента используется HTTP-only cookie с именем `access_token`. Токен никогда не передаётся в JavaScript-код браузера: cookie устанавливается серверным Next.js API-маршрутом `POST /api/auth/login` после успешного получения токена от бэкенда NestJS. При последующих запросах клиентский код Next.js не работает с токеном напрямую — вместо этого все обращения к бэкенду идут через серверные маршруты `app/api/`, которые читают cookie из запроса и добавляют заголовок `Authorization: Bearer ...` перед перенаправлением запроса на бэкенд.

Такой подход обеспечивает защиту от нескольких типов атак. Во-первых, злоумышленник не может украсть токен через XSS-уязвимость: даже при внедрении произвольного JavaScript-кода на страницу он не сможет прочитать HTTP-only cookie. Во-вторых, клиентский код не содержит токен в DOM, переменных или `localStorage`, что уменьшает поверхность атаки.

3.4.4 Ролевая модель доступа

В системе реализована ролевая модель с тремя уровнями доступа: `USER`, `MODERATOR` и `ADMIN`. Роль сохраняется в поле `role` таблицы `User` и передаётся в составе JWT-токена. Проверка роли выполняется в контроллерах при выполнении привилегированных операций:

1. Обычный пользователь может создавать публикации, редактировать и удалять собственные посты, добавлять посты в избранное и убирать их оттуда.
2. Модератор дополнительно имеет право удалять любые публикации для поддержания качества контента.
3. Администратор обладает полными правами: помимо возможностей модератора он может добавлять университеты и дисциплины в каталог.

Клиентская часть также учитывает роль пользователя при отображении интерфейса: кнопки «Добавить» на страницах каталога показываются только администраторам, а кнопки «Редактировать» и «Удалить» на странице поста — только автору публикации или пользователю с ролью модератора/администратора. Однако клиентские проверки носят вспомогательный характер: основная проверка прав выполняется на сервере, что исключает возможность обхода ограничений путём прямого обращения к API.

3.4.5 Валидация входных данных

Все входные данные, поступающие от пользователя, проходят валидацию на сервере. Для полей формы проверяется наличие обязательных значений и соответствие типам; при нарушениях возбуждается `BadRequestException` со статусом 400. Для загружаемых файлов применяются дополнительные ограничения:

1. Проверка MIME-типа: принимаются только файлы, начинающиеся с `image/`. Файлы с другими типами отклоняются на уровне `fileFilter` библиотеки `multer`.
2. Ограничение максимального размера файла и количества файлов в одном запросе (до пяти изображений для публикации).
3. Генерация уникальных имён файлов по шаблону `{timestamp}-{random}.{ext}`, что исключает коллизии и возможность перезаписи чужих файлов.

Такая валидация предотвращает загрузку вредоносных файлов (например, скриптов с расширением `.php` или `.js`) под видом изображений и защищает файловую систему сервера от переполнения.

3.4.6 Защита от SQL-инъекций

Все запросы к базе данных формируются через `Prisma Client`, который использует параметризованные запросы. Значения полей передаются в виде аргументов запроса, а не конкатенируются в строку SQL, что исключает возможность внедрения постороннего SQL-кода через пользовательский ввод. Декларативная схема базы данных и типобезопасный клиент обеспечивают дополнительный уровень защиты: невозможно сформировать некорректный запрос, используя поля, отсутствующие в модели.

3.4.7 Защита от XSS

Поскольку `React` по умолчанию экранирует все строковые значения при рендеринге, базовая защита от XSS-атак обеспечивается на уровне фреймворка. Пользовательский текст (заголовки и тела публикаций) выводится в компонентах без использования `dangerouslySetInnerHTML`, что исключает интерпретацию HTML-кода из пользовательского ввода. Дополнительную защиту обеспечивает использование `HTTP-only cookies` для хранения токена, как описано выше.

3.4.8 Настройка CORS

Для ограничения возможных источников запросов к бэкенду `NestJS` в приложении настроен CORS: разрешены только запросы с домена фронтенда (`http://localhost:3000` в среде разработки). Это предотвращает выполнение запросов к API из сторонних доменов и защищает пользователей от атак типа CSRF в сочетании с использованием `HTTP-only cookies`.

Таким образом, в системе ConSuccess реализован комплекс мер информационной безопасности, охватывающих аутентификацию, авторизацию, валидацию входных данных, защиту от распространённых уязвимостей веб-приложений и ограничение поверхности атаки. Такой подход обеспечивает защиту как пользовательских данных, так и внутренней инфраструктуры системы.

3.5 Масштабируемость и перспективы развития

Текущая реализация веб-приложения ConSuccess покрывает ключевые сценарии обмена учебными материалами и обеспечивает работоспособность базового функционала: иерархический каталог, публикацию постов с изображениями, раздел «Знания», авторизацию и механизм избранного. Тем не менее, в перспективе развития платформы можно выделить несколько направлений, направленных на расширение возможностей, повышение производительности и улучшение пользовательского опыта.

3.5.1 Расширение каталога

Одним из первых шагов масштабирования является расширение базового классификатора. В текущей реализации каталог построен на иерархии «университет → дисциплина → публикация». В дальнейшем могут быть добавлены следующие уровни и атрибуты:

1. Тип материала (конспект, лабораторная работа, задача, методические указания, пример кода), что позволит пользователям фильтровать контент не только по дисциплине, но и по характеру материала.
2. Теги — свободные метки, проставляемые автором публикации. Теги могут дополнять формальную классификацию и использоваться для семантического поиска.
3. Учебный семестр и год публикации — атрибуты, позволяющие отслеживать актуальность материалов и различать версии, относящиеся к разным периодам обучения.
4. Преподаватель — привязка публикаций к конкретному преподавателю, что полезно в случаях, когда одна и та же дисциплина ведётся разными преподавателями с разной программой.

Расширение схемы базы данных требует добавления новых таблиц и миграций Prisma, а также обновления форм публикации и страниц каталога. Поскольку Prisma поддерживает инкрементальные миграции, эти изменения могут быть внесены поэтапно, без нарушения работы существующей части системы.

3.5.2 Полнотекстовый поиск

В текущей реализации навигация по каталогу построена исключительно на иерархии «университет → дисциплина → публикация». Для большого каталога такой подход становится недостаточным: пользователь может знать ключевые слова искомого материала, но не помнить, к какой именно дисциплине он относится. Решением является внедрение полнотекстового поиска.

PostgreSQL предоставляет встроенные возможности полнотекстового поиска через тип данных `tsvector` и оператор `@@`. Для русского языка доступны словари-поисковики с поддержкой морфологии. Интеграция в Prisma осуществляется через `unsupported` поля или прямые SQL-запросы через `prisma.$queryRaw`. Альтернативой является использование специализированных поисковых движков, таких как Meilisearch или Elasticsearch, которые обеспечивают индексацию и ранжирование результатов.

Пользовательский интерфейс поиска может быть реализован в виде поисковой строки в шапке приложения, открывающей страницу результатов с фильтрами по университету, дисциплине, типу материала и тегам. Результаты поиска можно дополнительно ранжировать по популярности и дате публикации.

3.5.3 Модерация и система жалоб

С ростом количества пользователей увеличивается риск появления нежелательного контента: публикаций, нарушающих правила платформы, материалов с неточной информацией или материалов, защищённых авторским правом. В текущей реализации модерация осуществляется в ручном режиме пользователями с ролями MODERATOR и ADMIN, которые могут удалять любые публикации. В дальнейшем предлагается внедрить систему жалоб:

1. Кнопка «Пожаловаться» на странице поста с возможностью выбора причины (некорректное содержимое, нарушение авторских прав, дубликат, спам).
2. Очередь жалоб для модераторов с возможностью просмотра, принятия решения и обратной связи с автором публикации.
3. Автоматическая блокировка публикации при превышении порога числа жалоб, с последующей проверкой модератором.

Данный механизм повышает прозрачность модерации и снижает нагрузку на модераторов, поскольку сообщество само участвует в выявлении проблемного контента.

3.5.4 Система рекомендаций

Для повышения вовлечённости пользователей может быть внедрена система рекомендаций. В простейшем варианте она может основываться на следующих принципах:

1. Популярность: публикации с наибольшим числом добавлений в избранное выводятся на главной странице.
2. Активность дисциплины: дисциплины с большим количеством свежих публикаций получают приоритет в списках.

3. Коллаборативная фильтрация: пользователям, добавившим одни и те же публикации в избранное, показываются аналогичные материалы.

Реализация рекомендаций требует ведения статистики просмотров и добавлений в избранное, а также периодического пересчёта рекомендованных подборок.

3.5.5 Мобильная версия и адаптивность

В текущей реализации интерфейс ориентирован на настольные устройства. Дальнейшее развитие предполагает адаптацию интерфейса для мобильных устройств с учётом особенностей сенсорных экранов: выпадающее меню вместо боковой панели, крупные области нажатия, оптимизация галереи для свайпов. Поскольку Tailwind CSS поддерживает адаптивные варианты стилей (sm:, md:, lg:), адаптация может быть выполнена без существенной реструктуризации компонентов.

3.5.6 Кэширование и производительность

По мере роста числа пользователей и объёма данных потребуется оптимизация производительности. Возможные направления:

1. Кэширование результатов запросов на стороне сервера через Redis. Часто запрашиваемые данные (список университетов, список дисциплин) могут храниться в кэше с ограниченным временем жизни, что снижает нагрузку на базу данных.
2. Кэширование на стороне клиента через TanStack Query с настраиваемыми стратегиями инвалидации. Это уменьшает количество повторных запросов при навигации между страницами.
3. Оптимизация изображений: вместо отдачи оригинальных файлов возможна генерация миниатюр разных размеров и использование современных форматов (WebP, AVIF). Для этого можно интегрировать библиотеку sharp на стороне сервера.
4. CDN для статических ресурсов (логотипы университетов, вложения публикаций), что ускоряет доставку контента географически удалённым пользователям.

3.5.7 Развёртывание и масштабирование инфраструктуры

Для продуктивной эксплуатации системы потребуется переход от локальной разработки к полноценному развёртыванию. Возможные варианты:

1. Контейнеризация приложения через Docker с отдельными образами для фронтенда, бэкенда и базы данных. Это упрощает развёртывание и обеспечивает воспроизводимость среды.
2. Размещение базы данных PostgreSQL на выделенном сервере или в облачном сервисе с настроенными резервными копиями и репликацией.

3. Вынесение хранилища загруженных файлов в объектное хранилище (например, Amazon S3 или его совместимые аналоги), что делает бэкенд stateless и упрощает горизонтальное масштабирование.
4. Использование балансировщика нагрузки перед несколькими экземплярами бэкенда и фронтенда для обеспечения отказоустойчивости и обработки возрастающего трафика.

3.5.8 Интеграция с внешними сервисами

Дальнейшая интеграция с внешними сервисами может расширить функциональность платформы. Например, поддержка входа через учётные записи социальных сетей или образовательных систем упростит регистрацию; интеграция с сервисами автоматического распознавания рукописного текста позволит преобразовывать фотографии конспектов в текст; добавление экспорта публикаций в формате PDF облегчит офлайн-использование материалов.

Таким образом, разработанная платформа обладает значительным потенциалом для дальнейшего развития. Модульная архитектура, типобезопасная работа с базой данных и чёткое разделение ответственности между клиентом и сервером создают основу для поэтапного расширения функциональности без необходимости переработки существующего кода. Перечисленные направления позволяют постепенно превратить ConSuccess из локального решения для обмена учебными материалами в полноценную образовательную платформу, поддерживающую широкий круг пользователей и разнообразные учебные сценарии.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы было разработано веб-приложение ConSuccess, предназначенное для коллективного обмена учебными материалами между студентами. Система решает актуальную задачу фрагментации учебных ресурсов, с которой сталкиваются студенты при подготовке к занятиям: неформальные каналы обмена (мессенджеры, облачные диски, личные папки) не обеспечивают единой структуры хранения, удобной навигации и механизмов повторного доступа к найденным материалам. Разработанная платформа предоставляет иерархический каталог «университет → дисциплина → публикация» с поддержкой текстовых постов, прикрепления изображений, раздела «Знания» для общих материалов и механизма избранного для формирования персональной базы знаний.

В рамках работы были решены следующие задачи:

1. Проведён анализ предметной области и выделены ключевые проблемы существующих способов обмена учебными материалами: отсутствие единой классификации,

ограниченные возможности поиска, отсутствие персонализации и потеря материалов при смене учебной группы.

2. Выполнен обзор существующих аналогов (Notion, YoNote, StuDocu, StudFiles) с выявлением их сильных и слабых сторон. Определены элементы, которые целесообразно перенять, а также ключевые отличия, необходимые для соответствия задаче.
3. Определены целевая аудитория и пользовательские роли (USER, MODERATOR, ADMIN) с распределением прав между ними.
4. Сформированы функциональные требования и критический путь для каждой роли, что стало основой для дальнейшего проектирования.
5. Спроектирована архитектура клиент-серверного веб-приложения с разделением ответственности между фронтендом на Next.js и бэкендом на NestJS.
6. Разработана реляционная модель данных с семью сущностями (User, City, University, Subject, Post, Attachment, Favorite), описана в декларативной схеме Prisma и развёрнута в PostgreSQL.
7. Реализован REST API на NestJS, включающий авторизацию через JWT, загрузку изображений через Multer, работу с базой данных через Prisma ORM и ролевую модель доступа.
8. Реализована клиентская часть на Next.js с иерархическим каталогом, формами публикации постов, галереей изображений с режимом лайтбокса, разделом «Знания», механизмом избранного и адаптивной навигацией.
9. Разработаны макеты ключевых страниц системы с единой компоновочной схемой и обоснованными проектными решениями по размещению навигационных элементов.
10. Составлены сценарии тестирования функциональных требований для всех трёх ролей, подтверждающие соответствие системы поставленным требованиям.
11. Реализован комплекс мер информационной безопасности: хеширование паролей bcrypt, авторизация через JWT, HTTP-only cookies, валидация загружаемых файлов, защита от SQL-инъекций через Prisma и защита от XSS на уровне React.
12. Описаны перспективные направления развития системы: расширение классификатора, внедрение полнотекстового поиска, система жалоб и модерации, адаптация под мобильные устройства, кэширование и оптимизация инфраструктуры.

Разработанная система обеспечивает решение поставленной цели — создание централизованной платформы для коллективного обмена учебными материалами,

организованной по учебным заведениям и дисциплинам. Применение современных технологий (Next.js, NestJS, PostgreSQL, Prisma, Tailwind CSS, TanStack Query) обеспечивает производительность, типобезопасность и удобство дальнейшей поддержки кодовой базы. Модульная архитектура и чёткое разделение серверного и клиентского слоёв создают основу для поэтапного расширения функциональности без необходимости переработки существующего кода.

Практическая значимость работы заключается в возможности использования разработанной платформы в студенческих сообществах вузов: система упорядочивает разрозненные учебные материалы, упрощает их поиск, обеспечивает безопасный доступ и формирует персональные подборки полезного контента. По сравнению с неформальными каналами обмена и существующими аналогами, ConSuccess сочетает в себе иерархический каталог, поддержку изображений, свободную публикацию без ограничений и механизм избранного, что делает её самодостаточным решением для учебной среды.

Перспективы дальнейшего развития включают внедрение полнотекстового поиска, расширение классификаторов (тип материала, теги, семестр, преподаватель), системы жалоб и модерации, интеграции с внешними сервисами и адаптацию под мобильные устройства. Эти направления позволяют постепенно превратить систему из локального решения в полноценную образовательную платформу, поддерживающую широкий круг пользователей и разнообразные учебные сценарии.

Таким образом, все поставленные задачи выполнены, цель работы достигнута. Разработанное веб-приложение готово к использованию в учебном сообществе и может служить основой для дальнейших исследований и практических проектов в области цифрового образования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. NestJS — A progressive Node.js framework. Официальная документация. — URL: <https://docs.nestjs.com/> (дата обращения: 01.04.2026).
2. Next.js Documentation. App Router. — URL: <https://nextjs.org/docs/app> (дата обращения: 01.04.2026).
3. Prisma ORM Documentation. Schema, migrations, client. — URL: <https://www.prisma.io/docs> (дата обращения: 02.04.2026).
4. PostgreSQL 16 Documentation. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 02.04.2026).
5. React — The library for web and native user interfaces. Официальная документация. — URL: <https://react.dev/> (дата обращения: 03.04.2026).
6. TypeScript Handbook. — URL: <https://www.typescriptlang.org/docs/> (дата обращения: 03.04.2026).
7. Tailwind CSS v4 Documentation. — URL: <https://tailwindcss.com/docs> (дата обращения: 04.04.2026).
8. shadcn/ui — Re-usable components built with Radix UI and Tailwind CSS. — URL: <https://ui.shadcn.com/> (дата обращения: 04.04.2026).
9. TanStack Query v4 Documentation. — URL: <https://tanstack.com/query/v4/docs/> (дата обращения: 05.04.2026).
10. React Hook Form Documentation. — URL: <https://react-hook-form.com/docs> (дата обращения: 05.04.2026).
11. JSON Web Tokens Introduction (RFC 7519). — URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 06.04.2026).
12. Multer — Node.js middleware for handling multipart/form-data. — URL: <https://github.com/expressjs/multer> (дата обращения: 06.04.2026).
13. bcrypt — A library to help you hash passwords. — URL: <https://github.com/kelektiv/node.bcrypt.js> (дата обращения: 07.04.2026).
14. Axios — Promise based HTTP client for the browser and node.js. — URL: <https://axios-http.com/docs/intro> (дата обращения: 07.04.2026).
15. OWASP Top Ten Project — Top 10 Web Application Security Risks. — URL: <https://owasp.org/www-project-top-ten/> (дата обращения: 08.04.2026).

16. MDN Web Docs. HTTP Cookies. — URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies> (дата обращения: 08.04.2026).
17. Fowler M. Patterns of Enterprise Application Architecture. — Addison-Wesley Professional, 2003. — 560 p.
18. Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. — Prentice Hall, 2017. — 432 p.
19. Newman S. Building Microservices: Designing Fine-Grained Systems. — O'Reilly Media, 2021. — 612 p.
20. Osmani A. Learning JavaScript Design Patterns. — O'Reilly Media, 2023. — 304 p.
21. Kleppmann M. Designing Data-Intensive Applications. — O'Reilly Media, 2017. — 616 p.
22. Грофф Дж., Вайнберг П. SQL. Полное руководство. 3-е изд. — М.: Вильямс, 2019. — 960 с.

СЦЕНАРИЙ ВИДЕОРОЛИКА ДЛЯ ЗАЩИТЫ ВКР

Длительность видеоролика — 3 минуты. Сценарий построен через решение проблемного кейса целевой аудитории; демонстрируются действия трёх ключевых ролей: гость, студент (USER) и администратор (ADMIN).

Кейс

Студент готовится к экзамену и не может найти нужные конспекты: материалы разбросаны по чатам, облачным дискам и личным папкам — часть ссылок устарела, часть файлов потеряна. Решение — ConSuccess: централизованная платформа обмена учебными материалами, структурированная по университетам и дисциплинам.

Хронометраж

0:00 – 0:30 — Проблема (роль: гость)

Закадровый текст описывает ситуацию: студент ищет конспект, но находит только устаревшие файлы и пустые ссылки в чатах. Демонстрируется главная страница ConSuccess — список последних публикаций из всех разделов как альтернативное решение.

0:30 – 1:15 — Просмотр каталога (роль: гость)

1. Открыть страницу «Университеты» — показать карточки с логотипами и названиями городов.
2. Выбрать университет → перейти к списку дисциплин.
3. Выбрать дисциплину → показать список постов с превью изображений и кратким текстом.
4. Открыть конкретный пост — прочитать полный текст, пролистать галерею изображений, открыть одно из них в полноэкранном режиме (lightbox).

1:15 – 2:10 — Публикация и избранное (роль: студент)

1. Войти в систему через форму входа (или предварительно зарегистрироваться).
2. Перейти к нужной дисциплине → нажать кнопку «Добавить пост».
3. Заполнить заголовок и текст публикации, прикрепить до пяти изображений → нажать «Добавить».
4. Убедиться, что новый пост появился в списке дисциплины в виде карточки.
5. Открыть пост другого пользователя → нажать кнопку-звёздочку «В избранное».
6. Показать боковую панель «Избранное» — пост отображается в списке сохранённых; перейти к нему по ссылке в один клик.
7. Вернуться к своей публикации → отредактировать текст → сохранить изменения.

2:10 – 2:45 — Управление каталогом (роль: администратор)

1. Войти в систему под учётной записью администратора.
2. Перейти на страницу «Университеты» → нажать «Добавить университет»: ввести название, выбрать город из списка, загрузить логотип.
3. Открыть созданный университет → добавить дисциплину.
4. Найти нежелательную публикацию любого пользователя → удалить её.

2:45 – 3:00 — Итог

Показать раздел «Знания» — общие публикации вне привязки к дисциплинам. Завершающий кадр: главная страница с последними постами. Закадровый текст: платформа ConSuccess решает проблему фрагментации учебных материалов — всё структурировано по университетам и дисциплинам, доступно в несколько кликов.